Goodix Health SDK Porting Guide

Revision: 0.1

Release Date: 2025-06-16

Goodix Confidential
Goodix Confidential For lierda_hd@lierda.com Only

Shenzhen Goodix Technology Co., Ltd.

Copyright

Copyright © 2025 Shenzhen Goodix Technology Co., Ltd. All rights reserved.

This document contains information that is proprietary to Shenzhen Goodix Technology Co.,Ltd. ('GOODIX'). The holder of this document shall treat all information contained herein as confidential, shall use the information only for its intended purpose, and shall not duplicate, disclose, or disseminate any of this information in any manner unless GOODIX has provided express, written permission otherwise.

Trademarks

"汇顶", "汇顶科技", "GOODiX" are trademarks or registered trademarks of Shenzhen Goodix Technology Co., Ltd. or its affiliates.

Disclaimer

This document conveys no express or implied licenses to any intellectual property rights belonging to GOODIX. GOODIX assumes no liability whatsoever for any use of the information contained in this document or for the consequences arising from such use by the holder of this document. Unless there is an express written agreement with GOODIX, GOODIX makes no warranties of non-infringement of intellectual property rights.

The information contained in this document is only provided for convenience to the holder of this document, and GOODIX may, from time to time and at its sole option, update the information contained in this document without notice. GOODIX makes no representations or warranties, express or implied, written or oral, legal or otherwise, with respect to the information contained in this document, including, but not limited to, any representations or warranties with respect to its use, quality, performance, merchantability or fitness for a particular purpose. The holder of this document shall ensure that the information contained in this document is used in accordance with technical specifications. Without an express, written permission provided by GOODIX, it is not permitted to use GOODIX products as critical components in life support systems.

Any source code (software and/or firmware) contained in this document is proprietary to GOODIX, and the source code may not be reproduced, modified, converted, compiled, or demonstrated without express written permission of GOODIX.

Any solution data if involved in this document pertain to design value or typical value, and any control value of mass production represents the control solution or data for current production line. Delivery standard is based on the current control value, and final definition after verification by both parties shall prevail.

Goodix Confidential For jing.hu@herda.com Only

Shenzhen Goodix Technology Co., Ltd.

Headquarters: 26F, Goodix Headquarters, No.1 Meikang Road, Futian District, Shenzhen, China; Zip Code:518000

TEL: +86-755-33338828

Website: http://www.goodix.com

Preface

Purpose

This document is applicable for Goodix Health sensor.

Audience

This document is intended for:

- Software developers
- Systems and product engineers
- Technical writers

Release Notes

This document is the first release.

Revision History

Revision	Date	Description
1.0	2025-06-16	First release.

Goodix Confidential
Goodix Confidential for lierda_hd@lierda.com Only

Contents

1 Overview 1

1.1 Environment Requirements 1

1.1.1 Hardware Operating Environment 1

1.1.2 Software Operating Environment 1

1.2 Features 1

1.3 Example Code 1

1.4 Structure 2

2 SDK Introduction 1

2.1 SDK Scenarios 1

2.1.1 Standard process for use with Goodix PC tools 1

2.1.2 Standard process used by users 2

2.1.3 SDK minimum usage process 3

2.2 APP detailed 3

2.2.1 APP folder 4

2.2.2 gh_app folder 4

2.2.3 gh_algo folder 5

2.2.4 gh_protocol folder 6

2.3 HAL detailed 7

2.3.1 gh_hal folder 7

2.3.2 hw_interface folder 8

2.3.3 hw_service folder 8

3 SDK Routine Porting Steps 11

3.1 Porting HAL Layer Code 11

3.1.1 Add GH SDK files into Project 11

3.1.2 Modify Configuration File 11

3.1.3 Add User Code 13

3.1.4 HAL Layer Integration 16

3.2 Porting APP Layer Code 17

3.2.1 Add GH SDK files into Project 17

Goodix Confidential
Goodix Confidential For lierda_hu@lierda.com Only

3.2.2 Modify Configuration File 19
3.2.3 Add User Code 26
3.2.4 Call G-sensor(ACC+GYRO) Related API 27
3.2.5 APP Layer Integration 28
3.2.6 Threads Arrangement 32
3.3 Porting Wear Detection 32
3.3.1 Porting Steps 32
3.4 Porting Goodix Protocol 33
3.4.1 Porting Steps 33
3.4.2 BLE peripheral setting 33
4 Advanced Feature Interfaces 34
5 Appendix 36
5.1 Appendix 1: Frame Data Structure Definition 36
5.2 Appendix 2: Troubleshooting FAQs During Porting 37
5.2.1 Compilation Error 37
5.2.2 Link Error 37
5.2.3 Operation Exceptions 38
5.3 Appendix 3: I2C Device Address Configuration 39

Goodix Confidential For lierda_hd@lierda.com Only
Goodix Confidential

1 Overview

This document introduces how to port the Goodix Health (GH) software development kit (SDK), and explain internal structure and functions. (Hereinafter referred to as GH SDK)

Corresponding SDK version: V0.2.0.0

1.1 Environment Requirements

1.1.1 Hardware Operating Environment

The general operating environment is little endian MCU platform like ARM M4, which requires an I2C or SPI API.

1.1.2 Software Operating Environment

The software environment requires the C standard header file 'stdint.h'.

Note:

If users need to generate dynamic link libraries, such as Android .so files or Windows .DLL files, please manually remove the 'weak' attribute or modifier in GH SDK.

1.2 Features

The GH SDK integrates chip driver, chip data processing, application logic and health algorithm, providing software support for users to use the goodix health series chips, making it convenient for users to quickly import chip solutions into products.

GH SDK includes the chip control interface (start sampling, stop sampling), chip data analysis and fusion, health algorithm control and simple wearing application logic. Users do not need to pay attention to the internal working principles and details, and can directly call the interface, making the operation simpler.

Besides, GH SDK also has the following features:

1. Convenient data debugging. The user MCU communicates with the Goodix host computer (EVK/APP) to provide debugging sampling data during the development process.
2. Rich functions. A large number of APIs related to chip functions are encapsulated, such as software AGC, ADT processing, etc., which users can use directly.
3. Functions can be tailored.
4. Portability. Users can quickly port to their own platform based on the provided demo code.

1.3 Example Code

The GH Demo routine encapsulates the top-level API for the user, such as initialization, interrupt processing, starting and stopping sampling, etc. At the same time, the routine will separate the part that needs to be implemented by the user into a ".c" file and a ".h" configuration file. The user can open or close a certain function by modifying the macro definition in the configuration file, and without changing other parts of the code.

Goodix Confidential
Goodix Confidential For lierda_hd@lierda.com Only

1.4 Structure

GH SDK mainly consists of two parts: APP layer and HAL layer. The APP layer mainly involves chip applications, such as health data and Gsensor integration, health algorithm call, motion detection and wearing logic, etc. The HAL layer mainly involves chip operations, such as chip communication, interrupt processing, FIFO data analysis, data calibration, soft AGC, etc.

GH SDK also comes with an additional protocol component, which is used to package the data output by the SDK into a standard format, which can be uploaded to the GOODIX PC host tool for display and recording, making it easier for users to debug in the early stage.

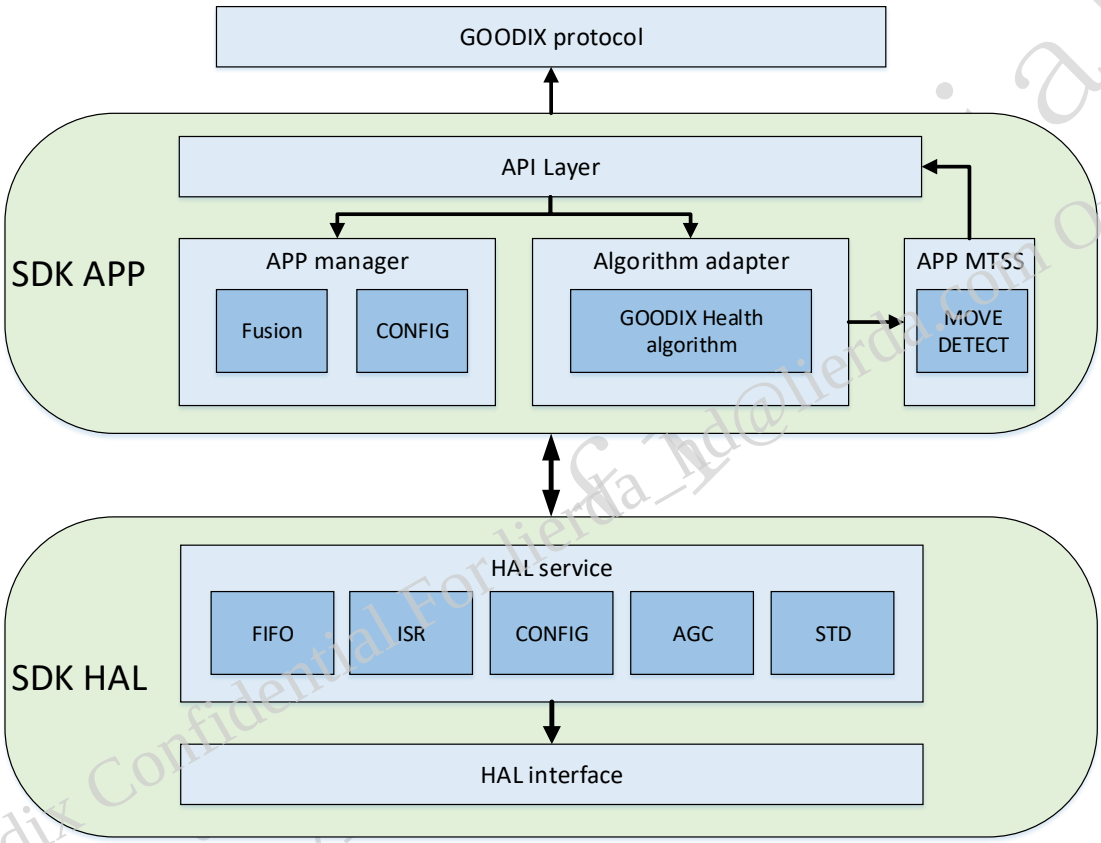


Figure 1-1 GH SDK structure

The file structure of GH SDK includes four folders: gh_app, gh_hal, gh_algo, and gh_protocol.

1. gh_app mainly stores the SDK APP layer code.
2. gh_hal mainly stores the SDK HAL layer code.
3. gh_algo mainly stores the GOODIX health algorithm.
4. gh_protocol mainly stores the GOODIX data packaging related code.

Figure 1-2 GH SDK folders

Goodix Confidential For lierda_hd@lierda.com Only

2 SDK Introduction

GH SDK is designed with the development needs of different users in mind, supporting both deep and light development.

Light users can directly use the entire GH SDK, including the HAL layer, APP layer, goodix health algorithm and wearing logic.

Deep development users have more choices:

1. For health algorithms, users can choose to replace each algorithm with a self-developed algorithm at will;

2. For wearing logic, users can design it independently;

3. Chip data and G-sensor data are integrated, and users can choose synchronous or asynchronous solutions, or develop it themselves;

4. For chip applications, users can only use the HAL layer to operate the chip, and all other related applications are implemented by themselves;

5. For users who require all codes to be self-developed, the HAL layer can be used as a reference code to facilitate users to operate the chip;

2.1 SDK Scenarios

2.1.1 Standard process for use with Goodix PC tools

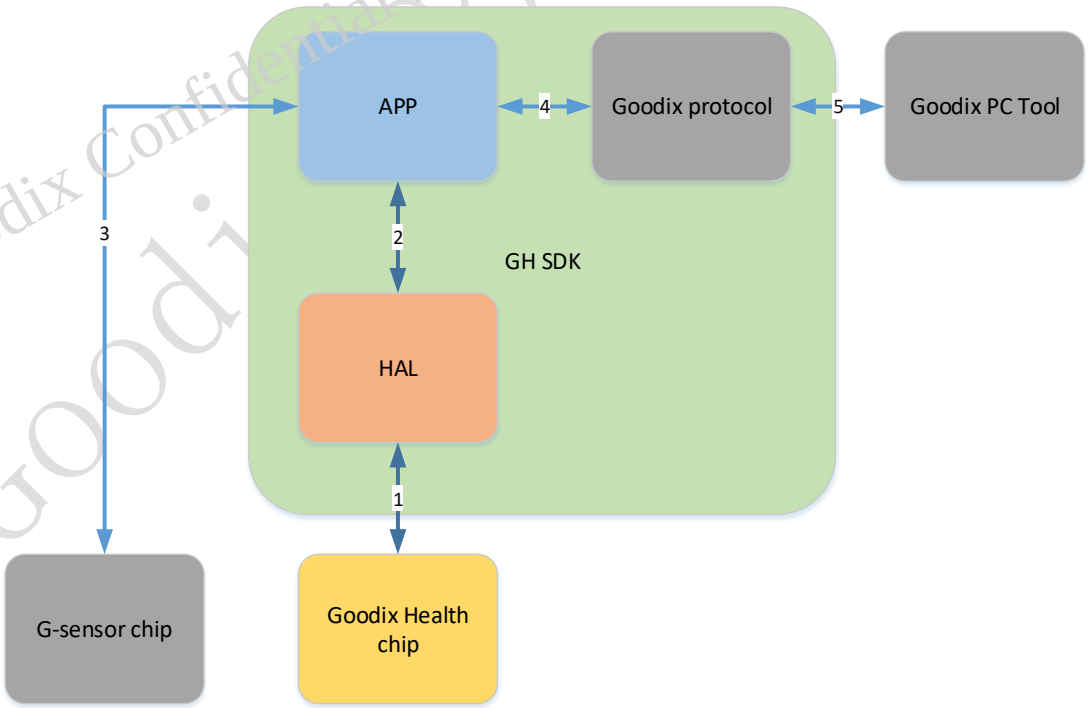


Figure 2-1 Standard process for use with Goodix PC tools

1. The application scenario of the GH Demo routine is shown in the figure above.

Goodix Confidential For lierda_hd@lierda.com only

2. MCU and GH Chip communicate through I2C/SPI. At the same time, there is an interrupt pin connection between the GH Chip and the MCU.
3. When the HAL layer submits the parsed chip data to the APP layer, it can directly call the corresponding interface in a single-threaded application or pass it through messages in a multi-threaded application according to user needs.
4. If the user requires the SDK to merge PPG data with G-sensor data, the G-sensor data can be provided to the SDK APP layer in a synchronous or asynchronous manner.
5. If users need to use Goodix PC Tool to view and save chip data and algorithm results in real time, they need to introduce the Goodix protocol module to let the SDK package the data according to the Goodix standard.
6. For point 4, users need to implement platform-related code in the Goodix protocol module, send data to the PC tool through the hardware interface (serial port, BLE), and parse the data sent by the PC tool to the Goodix protocol module.

2.1.2 Standard process used by users

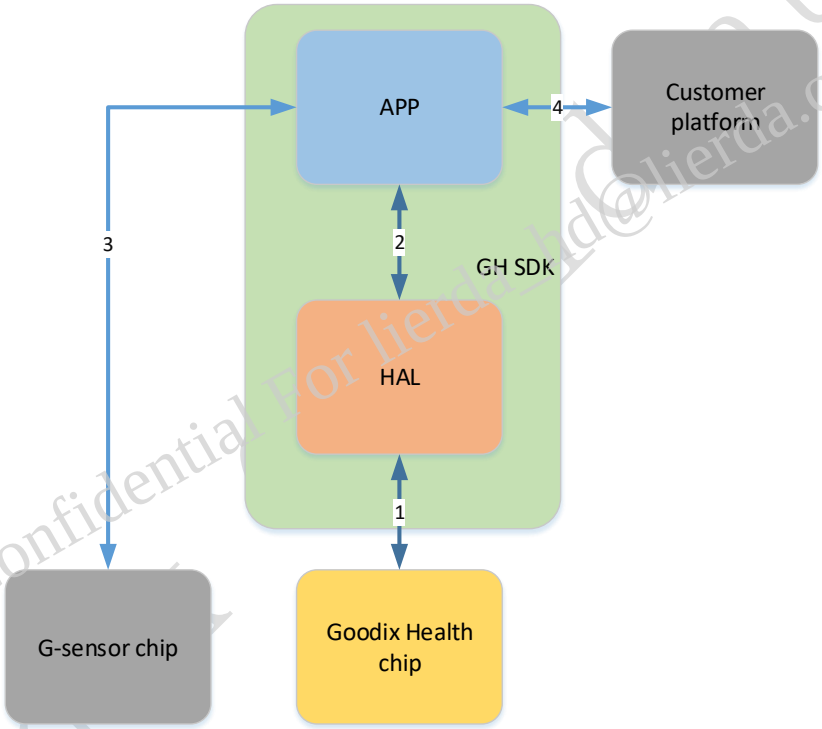


Figure 2-2 Standard process used by users

1. MCU and GH Chip communicate through I2C/SPI. At the same time, there is an interrupt pin connection between the GH Chip and the MCU.
2. When the HAL layer submits the parsed chip data to the APP layer, it can directly call the corresponding interface in a single-threaded application or pass it through messages in a multi-threaded application according to user needs.
3. If the user requires the SDK to merge PPG data with G-sensor data, the G-sensor data can be provided to the SDK APP layer in a synchronous or asynchronous manner.
4. Users obtain chip data and algorithm results after integration at the SDK APP layer. The data obtained by users from here has been packaged according to functions or algorithms.

Goodix Confidential For lierda_hd@lierda.com Only

2.1.3 SDK minimum usage process

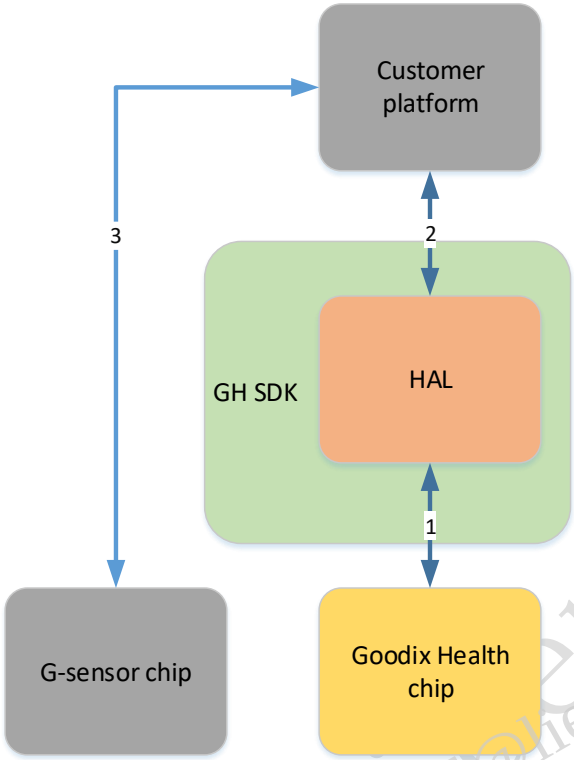


Figure 2-3 SDK minimum usage process

1. MCU and GH Chip communicate through I2C/SPI. At the same time, there is an interrupt pin connection between the GH Chip and the MCU.
2. Clients can obtain the original data of the fifo or the data packet after the fifo is parsed from here.
3. User-defined G-sensor data fusion solution

2.2 APP detailed

APP layer data processing standard process as follow:

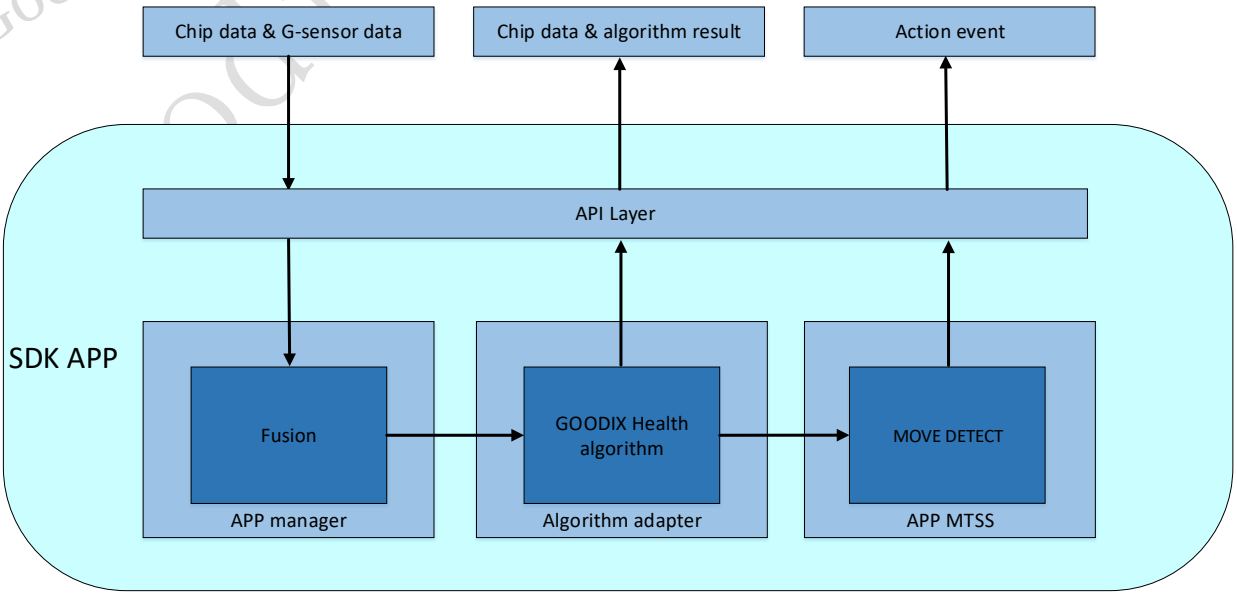


Figure 2-4 APP data processing process

Goodix Confidential For jerda_hd@jerda.com Only

2.2.1 APP folder

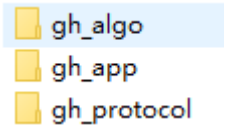


Figure 2-5 GH SDK APP folder

2.2.2 gh_app folder

The contents of the gh_app folder are analyzed as follows:

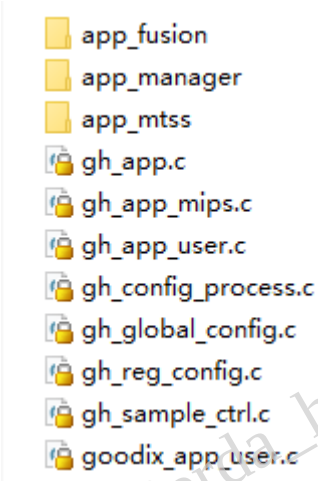


Figure 2-6 gh_app folder structure

Below is the file list details:

File name or folder name	Description	Can users modify or not
gh_fusion/ gh_data_fusion.c	Asynchronous solution for integrating chip data with G-sensor	No
gh_fusion/ gh_data_sync_fusion.c	Synchronization solution for chip data and G-sensor fusion	No
app_manager/ gh_algo_adapter.c	Algorithm adapter, used to manage the calls of all algorithm interfaces. Currently, the normalization driver of the GOODIX algorithm is in this file.	No
app_manager/ gh_algo_adt.c	Call the ADT algorithm interface. Currently, the default method is to call the GOODIX open source ADT algorithm.	Yes
app_manager/ gh_algo_hr_ third_party.c	Used for third-party HR algorithm interface calls	Yes
app_manager/ gh_algo_hrv_ third_party.c	Used for third-party HRV algorithm interface calls	Yes
app_manager/ gh_algo_mem.c	Used for dynamic memory application of GOODIX algorithm	No
app_manager/ gh_algo_nadt_	Used for third-party NADT algorithm interface	Yes

Goodix Confidential For lierda_hd@lierda.com Only

File name or folder name	Description	Can users modify or not
third_party.c	calls	
app_manager/ gh_algo_spo2_ third_party.c	Used for third-party SPO2 algorithm interface calls	Yes
app_manager/ gh_app_manager.c	Used for APP layer management, mainly responsible for application configuration analysis, algorithm adapter management and data fusion module management	No
app_mtss/ gh_mtss_evt_manager.c	The full name of MTSS is mutil-sensor, which is used to fuse multiple sensor data to determine the wearing status. Currently, it only supports the fusion of g-sensor data and PPG data to determine whether it is worn.	No
gh_app.c	The main logic file of the app layer, mainly used to call various components of the app layer	No
gh_app_mips.c	Used for algorithm computing power statistics	No
gh_app_user.c	It mainly includes the interface for the app to publish data externally and the platform-related interface required by the app. Users can obtain chip data and algorithm results from this file interface, as well as determine the interaction mode between the app layer and the hal layer.	Yes
gh_config_process.c	Used to distribute configuration to each component	No
gh_global_config.c	SDK global configuration, used to define customized SDK	Yes
gh_reg_config.c	SDK application configuration file, used to store the configuration generated by the PC tool	Yes
gh_sample_ctrl.c	Used to control the APP sampling interface	No
goodix_app_user.c	Used to adapt to GOODIX's DEMO and EVK platforms. The file interface is consistent with that in gh_app_user.c, for users to refer to when porting SDK	No

2.2.3 gh_algo folder

The contents of the gh_algo folder are analyzed as follows

Goodix Confidential For hierda_hd@hierda.com Only

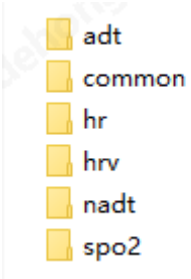


Figure 2-7 gh_algo folder structure

File name or folder name	Description	Can users modify or not
adt/ gh_move_detector.c	G-sensor motion detection algorithm source code	No
adt/ gh_wear_detector.c	Software wearing detection source code	No
common	Goodix algorithm public library	No
hr	Goodix heart rate algorithm library and configuration files	No
hrv	Goodix heart rate variability algorithm library and configuration files	No
nadt	Goodix native-ADT algorithm library and configuration file	No
spo2	Goodix Blood oxygen algorithm library and configuration files	No

2.2.4 gh_protocol folder

The contents of the gh_protocol folder are analyzed as follows:

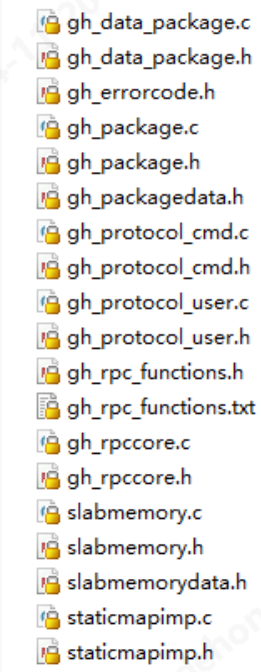


Figure 2-8 gh_protocol folder structure

Goodix Confidential For lierda.com Only

File name or folder name	Description	Can users modify or not
gh_protocol_user.c	This file contains the client platform interface implementation. After the SDK data is packaged according to the GOODIX standard, it is released through the interface in this file. For the data sent by the GOODIX PC tool, the interface in this file can be called for parsing.	Yes
Other files	The remaining files are all related to the GOODIX standard protocol and are used for chip and algorithm data packaging.	No

2.3 HAL detailed

The HAL layer includes hw_interface and hw_service. The hw_interface primarily serves as the communication interface between the sensor chip and the host controller, while the hw_service contains several modules: FIFO Parser, AGC, STD, and HAL Control. These modules are responsible for controlling the sensor chip and decode the raw data.

The APP layer can call HAL layer functions to control the sensor chip and retrieve chip data through callback functions.

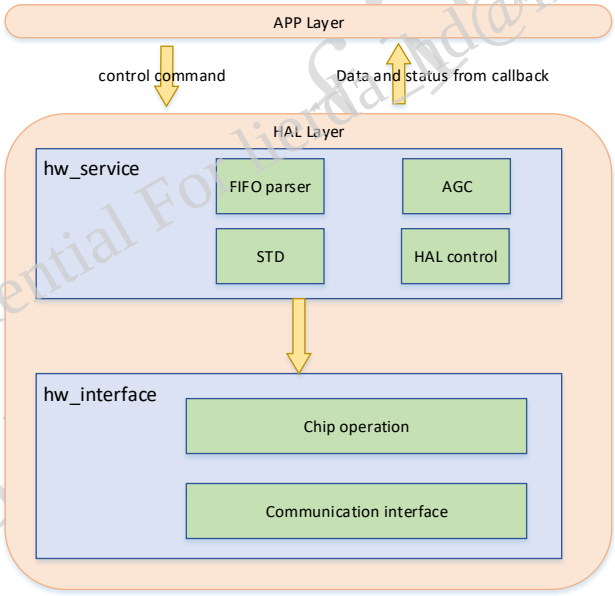


Figure 2-9 HAL Layer Structure

2.3.1 gh_hal folder

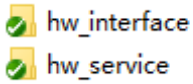


Figure 2-10 gh_hal folder

Goodix Confidential For jierda.hd@hierda.com only

2.3.2 hw_interface folder

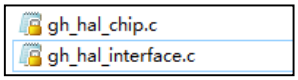


Figure 2-11 hw_interface folder

File name or folder name	Description	Can users modify or not
gh_hal_interface.c	This file implements the hardware abstraction layer (HAL) interface for different communication protocols such as SPI and I2C. It includes functions for reading and writing registers, handling FIFO operations, and managing low power modes. The file supports both software and hardware chip select (CS) for SPI communication and provides utility functions for endian swapping and delay handling.	No
gh_hal_chip.c	This file contains the implementation of hardware abstraction layer (HAL) functions for the GH chip. It includes functions for initializing the chip, configuring various parameters, reading and writing registers, and handling interrupts. It provides an interface for higher-level software to interact with the chip.	No

2.3.3 hw_service folder

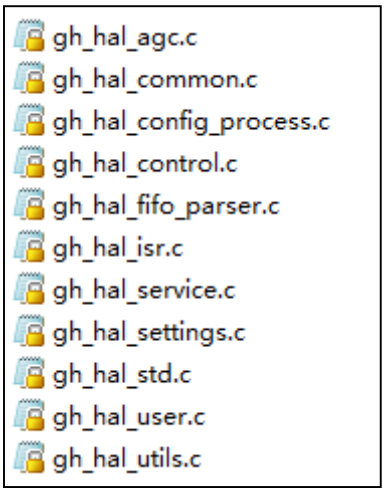


Figure 2-12 hw_service folder

File name or folder name	Description	Can users modify or not
gh_hal_agc.c	This file is part of AGC (Automatic Gain Control) implementation. It includes various functions and	No

Goodix Confidential For lierda_hd@lierda.com Only

File name or folder name	Description	Can users modify or not
	definitions for initializing, configuring, and processing AGC operations. The file handles tasks such as resetting AGC configurations, parsing register values, configuring virtual registers, and processing main and sub-channel AGC data.	
gh_hal_config_process	This file contains the implementation of hardware abstraction layer (HAL) configuration processes for a specific hardware service. It includes functions for configuring registers and downloading configurations.	No
gh_hal_control.c	This file contains the implementation of hardware abstraction layer (HAL) control functions for the GH hardware. It includes functions for initializing, resetting, starting, and configuring the control module, as well as enabling data channels and calculating FIFO watermark values.	No
gh_hal_fifo_parser.c	This file contains the implementation of various functions for parsing FIFO data in the GH HAL (Hardware Abstraction Layer). It includes functions for initializing the FIFO parser, registering callbacks, transforming and parsing different types of data, and managing the parser's status and configuration. The file also includes debug logging and support for different FIFO control modes.	No
gh_hal_isr.c	This file contains the implementation of interrupt service routines (ISRs) for the GH hardware abstraction layer (HAL). It includes functions for handling interrupts, and processing FIFO (First In, First Out) data.	No
gh_hal_service.c	This file implements the hardware abstraction layer (HAL) service for the GH system. It includes initialization, configuration, and control functions for various hardware components such as FIFO parser, controller, standard parser, and AGC module. The file also provides callback functions for data calibration and AGC processing.	No
gh_hal_settings.c	This file contains the implementation of functions for handling hardware settings. It includes initialization, resetting, configuration, and data count calculation for various hardware settings.	No
gh_hal_std.c	This file is part of the hardware abstraction layer (HAL) for standard operations. It includes various calibration and data processing functions for handling photoplethysmography (PPG) data, background (BG) data, and mixed data. The file also contains functions for setting LED driver codes and DC cancel codes, as well as initializing and resetting the parser configurations.	No
gh_hal_user.c	This file contains the implementation of various hardware abstraction layer (HAL) functions for the Goodix hardware.	Yes

Goodix Confidential
Goodix Confidential For herda_hu@herda.com Only

 Copyright ©2025 Shenzhen Goodix Technology Co., Ltd.

File name or folder name	Description	Can users modify or not
	These functions include timestamp retrieval, delay functions, logging, SPI and I2C communication, interrupt handling, and data publishing. The functions are defined as weak, allowing them to be overridden by user-defined implementations.	
gh_hal_utils.c	This file contains utility functions for memory operations, including memory copy, memory set, memory move, string length calculation, and memory comparison.	No

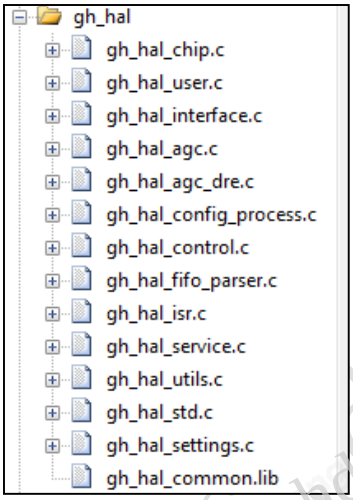
Goodix Confidential For lierda_hd@lierda.com Only
Goodix Confidential

3 SDK Routine Porting Steps

3.1 Porting HAL Layer Code

3.1.1 Add GH SDK files into Project

Add ".c" source files and “.lib” library files in “gh_hal” folder into project.



Add the following path to the include path of project:

1. gh_sdk\ gh_hal\ hw_interface\ inc
2. gh_sdk\ gh_hal\ hw_service\ inc

3.1.2 Modify Configuration File

Users can modify the macro definitions in the "gh_hal_config.h" file according to their needs. The configurable parameters are as follows.

Macro	Configurable Values	Description
GH_USE_SDK_APP	Defaults: 1 Optional: 0	1 means the user needs to use the APP layer of the SDK; 0 means the user does not need to use the APP layer of the SDK;
GH_INTERFACE_TYPE	GH_INTERFACE_I2C	Communication using I2C API
	GH_INTERFACE_SPI_SW_CS	Communication using software SPI API
	GH_INTERFACE_SPI_HW_CS	Communication using hardware SPI API
#if(GH_INTERFACE_TYPE == GH_INTERFACE_I2C)		
GH_I2C_DEVICE_ID_SEL	Defaults: GH_I2C_ID_SEL_1L0L Optional: GH_I2C_ID_SEL_1L0H GH_I2C_ID_SEL_1H0L GH_I2C_ID_SEL_1H0H	I2C address of the GH chip selects the pin level, please select the corresponding configuration macro according to the hardware schematic diagram. For details, refer to the Appendix 3: I2C Device Address Configuration

Goodix Confidential For lierda_hd@lierda.com Only

Macro	Configurable Values	Description
#endif		
GH_ISR_MODE	Defaults: INTERRUPT_MODE Optional: POLLING_MODE	For chip interrupt events, you can choose to process them by polling or by interrupt.
GH_FIFO_READ_BUFFER_SIZE	255 * 4 + GH_FIFO_BUFFER_EXTRA_LEN	The buffer size reserved for reading fifo, the fifo depth is 255 data packets, each data packet is 4 bytes, uers can set it according to the application to save memory
GH_FIFO_DATA_BULK_LEN	Defaults: 1 Optional: >=1	The number of data reported by the HAL layer at one time. The more data reported at one time, the greater the memory overhead.
GH_USE_STD_SNPRINTF	Defaults: 1 Optional: 0	1 means using snprintf in the C standard library; 0 means the client platform does not support standard snprintf, and the client needs to implement the function "int gh_hal_snprintf_user(char *p_str, size_t size, const char *p_format, ...)"
GH_SUPPORT_HARD_RESET	Defaults: 1 Optional: 0	1 means the client platform supports hard reset of GH chip; 0 means the client platform does not support hard reset of GH chip;
GH_SUPPORT_FIFO_CTRL_DEBUG0	Defaults: 0 Optional: 1	1 means the user needs chip debug0 type data; 0 means the user does not need chip debug0 type data; Setting to 0 can save memory
GH_SUPPORT_FIFO_CTRL_DEBUG1	Defaults: 0 Optional: 1	1 means the user needs chip debug1 type data; 0 means the user does not need chip debug1 type data; Setting to 0 can save memory
GH_SUPPORT_FIFO_CTRL_DRE_DC_INFO	Defaults: 1 Optional: 0	1 means the user needs chip DRE DC INFO data; 0 means the user does not need chip DRE DC INFO data; Setting to 0 can save memory
GH_HAL_STD_CALI_EN	Defaults: 1 Optional: 0	1 means turning on the SDK calibration function; 0 means turning off the SDK calibration function;
#if GH_HAL_STD_CALI_EN		
GH_HAL_STD_CALI_DRV_EN	Defaults: 1 Optional: 0	1 means enable drive current calibration; 0 means disable drive current calibration;
GH_HAL_STD_CALI_DC_CANCEL_EN	Defaults: 1 Optional: 0	1 means enable dc cancel calibration; 0 means disable dc cancel calibration;
GH_HAL_STD_CALI_BG_CANCEL_EN	Defaults: 1 Optional: 0	1 means enable bg cancel calibration; 0 means disable bg cancel calibration;
GH_HAL_STD_CALI_RX_OFFSET_EN	Defaults: 1 Optional: 0	1 means enable rx offset calibration; 0 means disable rx offset calibration;
GH_HAL_STD_CALI_GAIN_EN	Defaults: 1 Optional: 0	1 means enable gain calibration; 0 means disable gain calibration;

Goodix Confidential For lierda_hu@lierda.com only

Macro	Configurable Values	Description
GH_HAL_STD_CALI_RES_GAIN_EN	Defaults: 1 Optional: 0	1 means enable (dc cancel\bg cancel\rx offset) gain calibration; 0 means disable (dc cancel\bg cancel\rx offset) gain calibration;
GH_FIFO_USE_WATERMARK_LIMIT	Defaults: 0 Optional: 1	1 means that the number of FIFO data read does not exceed the watermark level.
GH_PARAM_BACKUP_EN	Defaults: 1 Optional: 0	1 indicates that the dimming parameters are reset to their initial values before each sampling starts.
GH_PARAM_SYNC_UPDATE_EN	Defaults: 1 Optional: 0	1 indicates using the flag for parameter synchronous updates to parse FIFO data.
GH_SUPPORT_SOFT_AGC	Defaults: 1 Optional: 0	1 indicates using software AGC.
#endif		

3.1.3 Add User Code

The code that user needs to add is stored in the “gh_hal_user.c” file, and the API of this part is divided into the following categories.

3.1.3.1 Log Function

In order to achieve rapid porting of GH SDK and problem location, it is recommended that user activate the log function first. The Log function needs to configure GH_LOG_DEBUG_ENABLE to 1, and implement the following corresponding functions:

GH_LOG_DEBUG_ENABLE Related Functions

Table 3-1 snpintf log Function

Function Name	gh_hal_log_user	
Description	Log print function	
Parameter in	char *str	Formatted strings, just use printf-like functions to print Such as: printf("%s", log_string)
Parameter out	None	
Return	None	

Note:

This method uses the standard C function snpintf, please check whether the user platform supports this function. If it is not supported, clients need to set GH_USE_STD_SNPRINTF to 0 and then implement the content of the gh_hal_snprintf_user function.

Function Name	gh_hal_snprintf_user	
Description	Log print function	
Parameter in	char *p_str, size_t size, const char *p_format, ...	Snprintf formatted strings
Parameter out	None	
Return	None	

Goodix Confidential For lierda_hd@lierda.com Only

3.1.3.2 Chip Read and Write Unit Operation Function

If the MCU and GH chip communicate through I2C API, user needs to configure "GH_INTERFACE_TYPE" to "GH_INTERFACE_I2C", and according to the hardware environment, configure the value of "GH_I2C_DEVICE_ID_SEL", which is the lower two digits of the I2C address. average value. In addition, users need to complete the implementation of the following APIs.

Table 3-2 I2C Operation Function

Function Name	Description	Parameter in		Parameter out	Return
gh_hal_i2c_init	I2C API initialization	None		None	None
gh_hal_i2c_read	I2C data read API	uint8_t device_id	I2C address	None	Error code 0: success
		const uint8_t write_buffer[]	Data write buffer		
		uint16_t write_length	Write data length(unit: byte)		
		uint8_t read_buffer[]	Data read buffer		
		uint16_t read_length	Read data length(unit: byte)		
gh_hal_i2c_write	I2C data write API	uint8_t device_id	I2C address	None	Error code 0: success
		const uint8_t write_buffer[]	Data write buffer		
		uint16_t length	Write data length (unit: byte)		

If the communication between the MCU and GH chip is through SPI API, user needs to configure "GH_INTERFACE_TYPE" to "GH_INTERFACE_SPI_SW_CS" and complete the implementation of the following APIs.

Table 3-3 SPI Operation Function

Function Name	Description	Parameter in		Parameter out	Return
gh_hal_spi_init	SPI API initialization	None		None	None
gh_hal_spi_write	SPI write data API (Note: In this function API, it must be ensured that the control of the CS pin of the SPI bus cannot be included, and this pin is only controlled by GH SDK.)	const uint8_t write_buffer[]	Data write buffer	None	Error code 0: success
		uint16_t length	Write data length(unit: byte)		
gh_hal_spi_read	SPI read data API (Note: In this function API, user does not need to implement the function of writing data, and must ensure that the control of the CS pin of the SPI bus cannot be included, and this pin is only controlled by GH SDK.)	uint8_t read_buffer[]	Data read buffer	None	Error code 0: success
		uint16_t length	Read data length(unit: byte)		
gh_hal_spi_cs_ctrl	SPI CS pin operation API (this function needs to be implemented when the software CS is enabled)	uint8_t cs_pin_level	CS pin level setting	None	None

The following table is the spi operation function that needs to be implemented during hardware CS operation, user needs to configure "GH_INTERFACE_TYPE" to "GH_INTERFACE_SPI_HW_CS" and complete the implementation of the following APIs

Goodix Confidential For Hierda_ldd@hierda.com Only

Table 3-4 SPI writr read Function

Function Name	gh_hal_spi_write_read	
Description	Spi write and read funciton	
Parameter in	uint8_t* tx_buffer	Data write buffer
	uint8_t* rx_buffer	Data read buffer
	uint16_t len	Read data length(unit: byte)
Parameter out	none	
Return	Error code 0: success	

If the MCU can operate the GH chip through the hardware reset pin, user needs to implement the configuration function of the reset pin (please set " GH_SUPPORT_HARD_RESET " to 1).

Table 3-5 Reset Operation Function

Function Name	Description	Parameter in		Parameter out	Return
gh_hal_reset_pin_init	Hardware reset pin initialization	None		None	None
gh_hal_reset_pin_ctrl	Reset pin level control function	uint8_t pin_level	Reset pin level setting	None	None

Users need to implement the configuration of GH chip interrupt pins if “GH_ISR_MODE” set to “INTERRUPT_MODE”.

Table 3-6 INT Operation Function

Function Name	gh_hal_int_pin_init
Description	Interrupt pin initialization
Parameter in	none
Parameter out	none
Return	none

In this function, user needs to initialize the interrupt pin, configure the interrupt trigger mode as rising edge trigger, and enable the interrupt of the pin.

3.1.3.3 Delay Function

Table 3-7 Delay Operation Function

Function Name	Description	Parameter in		Parameter out	Return
gh_hal_delay_us	Microsecond delay function	uint16_t usUsec	us	None	None
gh_hal_delay_ms	Millisecond delay function	uint16_t usMsec	ms	None	None

3.1.3.4 Timestamp function

If the user needs to use the fusion module of the app layer, the function content of the interface for obtaining the timestamp needs to be implemented.

Table 3-8 Timestamp function

Marco Name	Description	Parameter in	Parameter out	Return
gh_hal_get_timestamp	HAL layer obtains timestamp	None	None	Unix timestamp(Unit: ms)

Goodix Confidential For lierda_ld@lierda.com Only

3.1.4 HAL Layer Integration

This section demonstrates how to independently port the HAL layer and GH_USE_SDK_APP needs to be set to 0. If you need to port the APP layer, set GH_USE_SDK_APP to 1, and directly skip to Section 3.2.

3.1.4.1 Add Configuration Table

The configuration table is at the end of the configuration file generated by of GHealth EVK tool. Copy the configuration table into an array. If GH_USE_SDK_APP is set to 0, only copy registers with addresses less than 0x1000.

```
PPG_3Slot_2ADCs_FS=100Hz_RF=10k_INT=316us_BGR=2.config
addr, value, default
{0x0016,0x001f},// FASTEST_SAMPLE_RATE_DIVIDER:31,
{0x0020,0x2919},// FIFO_WATER_LINE:25, RG_FIFO_READ_INT_TI
{0x0380,0xf210},// RG_SLOT_INDEX0:0 : ppg_cfg_0, RG_SLOT_I
{0x038a,0x0093},// cfg0 SAMPLE_RATE_DIVIDER:9, cfg0 FIFO_C
{0x038c,0x0093},// cfg1 SAMPLE_RATE_DIVIDER:9, cfg1 FIFO_C
{0x038e,0x0093},// cfg2 SAMPLE_RATE_DIVIDER:9, cfg2 FIFO_C
{0x03aa,0x00a8},// cfg0 BG_LEVEL:2 level, cfg0 RX0_TIA:10
{0x03ac,0x00a8},// cfg1 BG_LEVEL:2 level, cfg1 RX0_TIA:10
{0x03ae,0x00a8},// cfg2 BG_LEVEL:2 level, cfg2 RX0_TIA:10
{0x03da,0x3500},// cfg0 *LED_SELECT:LED_0|LED_1,
{0x03dc,0x3a00},// cfg1 *LED_SELECT:LED_2|LED_3,
{0x03de,0x3000},// cfg2 *LED_SELECT:LED_4|LED_5,
{0x03ea,0x2626},// cfg0 DRV0_CURRENT(mA):5.96078, cfg0 DRV
{0x03ec,0x2626},// cfg1 DRV0_CURRENT(mA):5.96078, cfg1 DRV
{0x03ee,0x2626},// cfg2 DRV0_CURRENT(mA):5.96078, cfg2 DRV
{0x0484,0x0001},// RG_ADC_XP_SL:0:3.25MHz,
{0x0500,0x000b},// FIFO_UP_OVERFLOW_EN:1, FIFO_DOWN_OVERLO
{0x1006,0xbb6f},// timestamp low 16 bit  2024-10-14 14:34:
{0x1008,0x670c},// timestamp high 16 bit,
```

Figure 3-1 configuration file

```
gh_reg_t g_reg_array[] =
{
    {0x0020,0x2919},// FIFO_WATER_LINE:25, RG_FIFO_READ_INT_TIME
    {0x0380,0xf210},// RG_SLOT_INDEX0:0 : ppg_cfg_0, RG_SLOT_IND
    {0x038a,0x009f},// cfg0 SAMPLE_RATE_DIVIDER:9, cfg0 FIFO_CTR
    {0x038a,0x0093},// cfg0 SAMPLE_RATE_DIVIDER:9, cfg0 FIFO_CTR
    {0x038c,0x0093},// cfg1 SAMPLE_RATE_DIVIDER:9, cfg1 FIFO_CTR
    {0x038e,0x0093},// cfg2 SAMPLE_RATE_DIVIDER:9, cfg2 FIFO_CTR
    {0x03aa,0x00a8},// cfg0 BG_LEVEL:2 level, cfg0 RX0_TIA:10k,
    {0x03ac,0x00a8},// cfg1 BG_LEVEL:2 level, cfg1 RX0_TIA:10k,
    {0x03ae,0x00a8},// cfg2 BG_LEVEL:2 level, cfg2 RX0_TIA:10k,
    {0x03da,0x3500},// cfg0 *LED_SELECT:LED_0|LED_1,
    {0x03dc,0x3a00},// cfg1 *LED_SELECT:LED_2|LED_3,
    {0x03de,0x3000},// cfg2 *LED_SELECT:LED_4|LED_5,
    {0x03ea,0x2626},// cfg0 DRV0_CURRENT(mA):5.96078, cfg0 DRV1_
    {0x03ec,0x2626},// cfg1 DRV0_CURRENT(mA):5.96078, cfg1 DRV1_
    {0x03ee,0x2626},// cfg2 DRV0_CURRENT(mA):5.96078, cfg2 DRV1_
    {0x0484,0x0001},// RG_ADC_XP_SL:0:3.25MHz,
    {0x0500,0x000b},// FIFO_UP_OVERFLOW_EN:1, FIFO_DOWN_OVERLOWE
};
```

Figure 3-2 configuration table

3.1.4.2 HAL Layer Main Process

1. When the main program starts, call function “gh_hal_service_init” for initialization, call function “gh_hal_service_cfg_download” to configure the sensor chip, and call function “gh_sevice_demo_ctrl” to start sampling.
2. If GH_ISR_MODE is set to INTERRUPT_MODE, wait for the sensor chip to trigger an interrupt, and then call function “gh_hal_isr” to handle the interrupt in the main loop.
3. If GH_ISR_MODE is set to POLLING_MODE, poll and call function "gh_hal_isr".
4. Call a function “gh_sevice_demo_ctrl” to turn off sampling as needed.
5. The functions in the HAL layer are not thread-safe and require locking when called in multi-threaded scenarios.

Goodix Confidential For Hierda_hd@hierda.com only

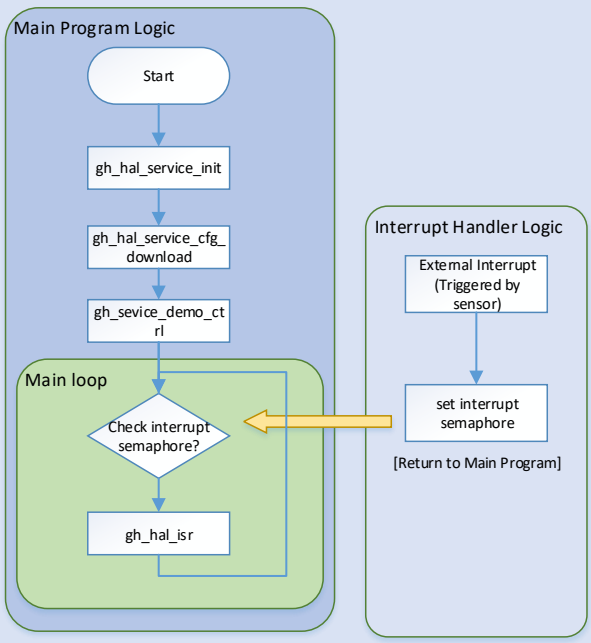


Figure 3-3 Main Process(Interrupt mode)

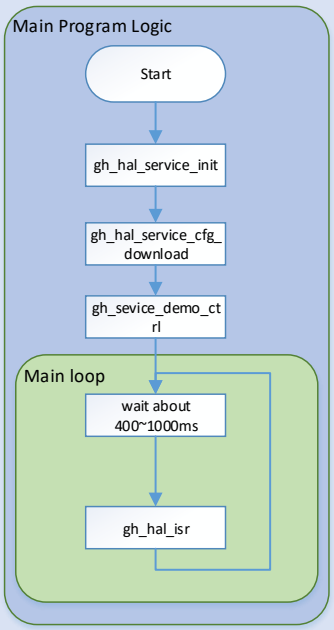


Figure 3-4 Main Process(polling mode)

3.1.4.3 Data and Status from Callback

Users can obtain raw data from two callbacks.

1. The function “gh_hal_fifo_data_publish” retrieves the raw FIFO data, which is a “uint8_t” array and requires further parsing by the user.
2. The function “gh_data_get_callback” provides data parsed by the HAL layer code, which is a “gh_data_t” array. “gh_data_t” is a union, and the data must be interpreted based on the data type. For detailed guidance, refer to the function “gh_data_get_callback” code examples and comments.

3.2 Porting APP Layer Code

The APP layer depends on the HAL layer. Before porting the APP layer, please ensure that the HAL layer code has already been ported.

3.2.1 Add GH SDK files into Project

Add ".c" source files in “gh_app” folder into project, and add ".c" source files and “.lib” library files in “gh_algo” folder into project.

Goodix Confidential For lierda hd@lierda.com Only

 Copyright ©2025 Shenzhen Goodix Technology Co., Ltd.

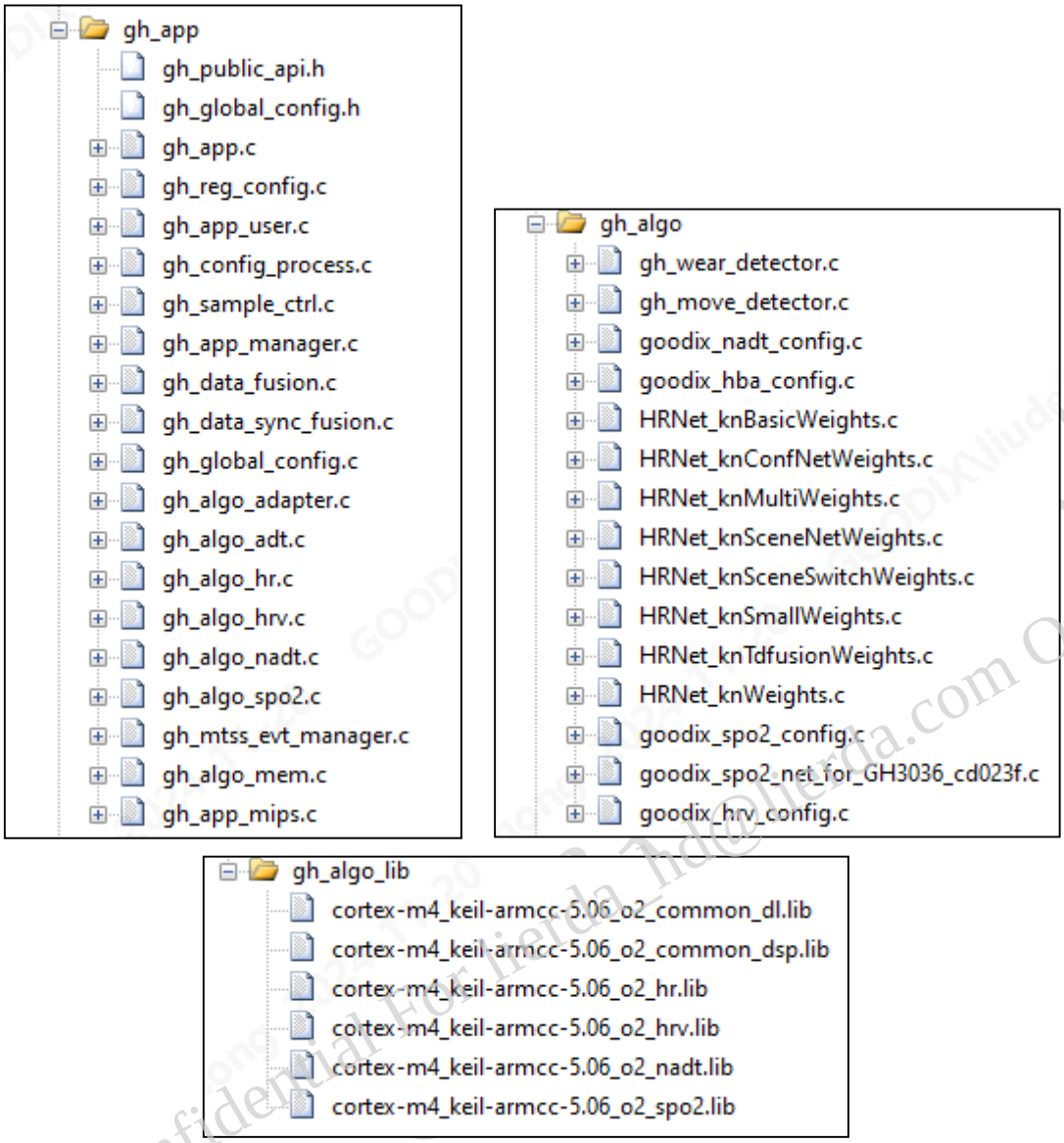


Figure 3-5 GH SDK File(Keil platform)

If users need to use Goodix PC tools, Add ".c" source files in “gh_protocol”

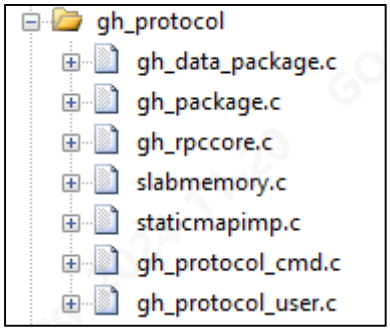


Figure 3-6 GH SDK protocol File(Keil platform)

Add the following path to the include path of project:

1. gh_sdk\ gh_app\ inc
2. gh_sdk\ gh_algo\ adt

Copyright ©2025 Shenzhen Goodix Technology Co., Ltd.

3. gh_sdk\ gh_algo\ common
4. gh_sdk\ gh_algo\ hr
5. gh_sdk\ gh_algo\ hrv
6. gh_sdk\ gh_algo\ nadt
7. gh_sdk\ gh_algo\ spo2

if users need to use Goodix PC tools,

8. gh_sdk\ gh_protocol

3.2.2 Modify Configuration File

This section explains how to configure the GH SDK. User can modify the macro definitions in the " gh_global_config.h" file according to their needs. The configurable parameters are as follows.

APP configuration is mainly in the "gh_global_config.h" file.

Table 3-9 Application Configuration Table

Macro	Configurable Values	Description
SDK basic function configuration		
GH_CHIP_TYPE	Defaults: GH_CHIP_3036 Optional: GH_CHIP_3018 GH_CHIP_3X2X GH_CHIP_33XX	Currently only supports GH_CHIP_3036
GH_FUNC_ADT_EN	Defaults: 1 Optional: 0	1 means that the software ADT function is required; 0 means that the software ADT function is not required
GH_FUNC_HR_EN	Defaults: 1 Optional: 0	1 means that the heart rate function is required; 0 means that the heart rate function is not required
GH_FUNC_SPO2_EN	Defaults: 1 Optional: 0	1 means blood oxygen function is needed; 0 means blood oxygen function is not needed
GH_FUNC_HRV_EN	Defaults: 1 Optional: 0	1 means that the heart rate variability function is needed; 0 means that the heart rate variability function is not needed
GH_FUNC_GNADT_EN	Defaults: 1 Optional: 0	1 means that the green light liveness detection function is required; 0 means that the green light liveness detection function is not required
GH_FUNC_IRNADT_EN	Defaults: 1 Optional: 0	1 means the red light liveness detection function is required; 0 means the red light liveness detection function is not required
GH_FUNC_TEST1_EN	Defaults: 1 Optional: 0	1 means that the custom 1 function needs to be reserved; 0 means that the custom 1 function does not need to be reserved
GH_FUNC_TEST2_EN	Defaults: 1 Optional: 0	1 means that the custom 2 function needs to be reserved; 0 means that the custom 2 function does not need to be reserved
GH_FUNC_SLOT_EN	Defaults: 1 Optional: 0	1 means pure data function is needed; 0 means pure data function is not needed; Pure data function means data does not need to be sent to any algorithm for processing

Goodix Confidential For lierda_hd@lierda.com Only

GH_ALGO_ADAPTER_EN	Defaults: 1 Optional: 0	1 means a health algorithm adapter is required; 0 means a health algorithm adapter is not required;
GH_ALGO_EN	Defaults: 1 Optional: 0	1 means a health algorithm is required; 0 means a health algorithm is not required;
End		
Algorithm switch configuration		
#if (1 == GH_ALGO_EN)		
GH_ALGO_ADT_EN	Defaults: 1 Optional: 0	1 means that the software ADT algorithm is required; 0 means that the software ADT algorithm is not required
GH_ALGO_HR_EN	Defaults: 1 Optional: 0	1 means that the heart rate algorithm is required; 0 means that the heart rate algorithm is not required
GH_ALGO_SPO2_EN	Defaults: 1 Optional: 0	1 means blood oxygen algorithm is needed; 0 means blood oxygen algorithm is not needed
GH_ALGO_HRV_EN	Defaults: 1 Optional: 0	1 means that the heart rate variability algorithm is needed; 0 means that the heart rate variability algorithm is not needed
GH_ALGO_GNADT_EN	Defaults: 1 Optional: 0	1 means that the green light liveness detection algorithm is required; 0 means that the green light liveness detection algorithm is not required
GH_ALGO_IRNADT_EN	Defaults: 1 Optional: 0	1 means that the red light liveness detection algorithm is required; 0 means that the red light liveness detection algorithm is not required
#endif		
#if (1 == GH_ALGO_ADT_EN)		
GH_USE_GOODIX_ADT_ALGO	Defaults: 1 Optional: 0	1 means using Goodix's software ADT algorithm; 0 means not using Goodix's software ADT algorithm;
#endif		
#if (1 == GH_ALGO_HR_EN)		
GH_USE_GOODIX_HR_ALGO	Defaults: 1 Optional: 0	1 means using Goodix's heart rate algorithm; 0 means not using Goodix's heart rate algorithm;
#endif		
#if (1 == GH_ALGO_HRV_EN)		
GH_USE_GOODIX_HRV_ALGO	Defaults: 1 Optional: 0	1 means using Goodix's heart rate variability algorithm; 0 means not using Goodix's heart rate variability algorithm;
#endif		
#if (1 == GH_ALGO_SPO2_EN)		
GH_USE_GOODIX_SPO2_ALGO	Defaults: 1 Optional: 0	1 means using Goodix's blood oxygen algorithm; 0 means not using Goodix's blood oxygen algorithm;
#endif		
#if (1 == GH_ALGO_GNADT_EN) || (1 == GH_ALGO_IRNADT_EN)		
GH_USE_GOODIX_NADT_ALGO	Defaults: 1 Optional: 0	1 means using Goodix's liveness detection algorithm; 0 means not using Goodix's liveness detection algorithm;
#endif		
GH_HR_ALGO_SUPPORT_CHNL_MAX	Defaults: 1 Optional: 1~4	The maximum number of channels for the heart rate algorithm. Each channel contains a set of green light, red light, and infrared data. The more channels, the greater the memory overhead.

Goodix Confidential For lierda_hd@lierda.com only

GH_SPO2_ALGO_SUPPORT_CHNL_MAX	Defaults: 1 Optional: 1~4	The maximum number of channels for the blood oxygen algorithm. Each channel contains a set of green light, red light, and infrared data. The more channels, the greater the memory overhead.
End		
The following are SDK scene presets, a total of 5 scenes, the SDK will automatically calculate the maximum memory required for all scenes.		
GH_ALGO_FUNCTION_GROUP_0	Defaults: (GH_FUNCTION_HR | GH_FUNCTION_SPO2 | GH_FUNCTION_HRV | GH_FUNCTION_NADT)	SDK runs scenario preset 0, which can run four algorithms at the same time: heart rate, blood oxygen, heart rate variability, and liveness detection.
GH_ALGO_FUNCTION_GROUP_1	Defaults: (GH_FUNCTION_SPO2)	SDK runs scene preset 1, runs the blood oxygen algorithm alone.
GH_ALGO_FUNCTION_GROUP_2	Defaults: GH_FUNCTION_HRV	The SDK runs scenario preset 2 and runs the heart rate variability algorithm alone.
GH_ALGO_FUNCTION_GROUP_3	Defaults: GH_NO_FUNCTION	SDK runs in scene preset 3, idle, no algorithm.
GH_ALGO_FUNCTION_GROUP_4	Defaults: GH_NO_FUNCTION	SDK runs in scene preset 4, idle, no algorithm.
End		
Algorithm version configuration		
GOODIX_HR_ALGO_VERISON	Defaults: GOODIX_ALGO_EXCLUSIVE Optional: GOODIX_ALGO_BASIC GOODIX_ALGO_MEDIUM GOODIX_ALGO_PREMIUM GOODIX_ALGO_EXCLUSIVE	Goodix heart rate algorithm version selection
GOODIX_SPO2_ALGO_VERISON	Defaults: GOODIX_ALGO_EXCLUSIVE Optional: GOODIX_ALGO_BASIC GOODIX_ALGO_MEDIUM GOODIX_ALGO_PREMIUM GOODIX_ALGO_EXCLUSIVE	Goodix blood oxygen algorithm version selection
Note: GOODIX_ALGO_BASIC and GOODIX_ALGO_MEDIUM are small versions, GOODIX_ALGO_PREMIUM and GOODIX_ALGO_EXCLUSIVE are large versions, and the blood oxygen and heart rate algorithms need to choose the same size version.		
End		
Algorithm memory application method configuration		
GH_USE_DYNAMIC_ALGO_MEM	Defaults: 0 Optional: 1	1 means that the algorithm memory needs to be dynamically allocated; 0 means that the algorithm memory can be statically allocated;
#if (1 == GH_USE_DYNAMIC_ALGO_MEM)		
GH_MALLOC	Defaults: malloc	The algorithm dynamically applies for memory interface
GH_FREE	Defaults: free	Algorithm releases dynamic memory interface

Goodix Confidential For lierda_hd@lierda.com Only

#endif		
End		
Data source configuration for each function		
GH_ADT_CONFIG_MAX	Defaults: GH_CHIP_PPG_CONFIG_NUM	The maximum number of slots required for the ADT function
GH_HR_CONFIG_MAX	Defaults: GH_CHIP_PPG_CONFIG_NUM	The maximum number of slots required for the HR function
GH_SPO2_CONFIG_MAX	Defaults: GH_CHIP_PPG_CONFIG_NUM	The maximum number of slots required for the SPO2 function
GH_HRV_CONFIG_MAX	Defaults: GH_CHIP_PPG_CONFIG_NUM	The maximum number of slots required for the HRV function
GH_NADT_CONFIG_MAX	Defaults: GH_CHIP_PPG_CONFIG_NUM	The maximum number of slots required for the NADT function
GH_TEST_CONFIG_MAX	Defaults: GH_CHIP_PPG_CONFIG_NUM	The maximum number of slots required for the TEST function
Note: Defined according to the actual needs of each algorithm, which can effectively reduce memory overhead		
GH_ADT_CONFIG_RX_MAX	Defaults: GH_CHIP_PPG_CONFIG_RX_NUM	The maximum number of RX required for the ADT function
GH_HR_CONFIG_RX_MAX	Defaults: GH_CHIP_PPG_CONFIG_RX_NUM	The maximum number of RX required for the HR function
GH_SPO2_CONFIG_RX_MAX	Defaults: GH_CHIP_PPG_CONFIG_RX_NUM	The maximum number of RX required for the SPO2 function
GH_HRV_CONFIG_RX_MAX	Defaults: GH_CHIP_PPG_CONFIG_RX_NUM	The maximum number of RX required for the HRV function
GH_NADT_CONFIG_RX_MAX	Defaults: GH_CHIP_PPG_CONFIG_RX_NUM	The maximum number of slots required for the NADT function
GH_TEST_CONFIG_RX_MAX	Defaults: GH_CHIP_PPG_CONFIG_RX_NUM	The maximum number of RX required for the TEST function
Note: The number of RX in each slot is fixed and can be selected according to the actual needs of the algorithm.		
GH_ADT_DATA_TYPE	Defaults: GH_CHIP_PPG_DATA0	Select what type of chip data the ADT algorithm requires. By default, only the chip's PPG data is required.

Goodix Confidential For herda_hd@herda.com Only

	Optional: GH_CHIP_PPG_DATA1 GH_CHIP_PPG_DEBUG0 GH_CHIP_PPG_DEBUG1_BGDATA0 GH_CHIP_PPG_DEBUG1_BGDATA1 GH_CHIP_PPG_DEBUG1_BGDATA2 GH_CHIP_PPG_DEBUG2_DRE GH_CHIP_CAP_RAWDATA GH_CHIP_PPG_DC_INFO	
GH_HR_DATA_TYPE	Defaults: GH_CHIP_PPG_DATA0 Optional: GH_CHIP_PPG_DATA1 GH_CHIP_PPG_DEBUG0 GH_CHIP_PPG_DEBUG1_BGDATA0 GH_CHIP_PPG_DEBUG1_BGDATA1 GH_CHIP_PPG_DEBUG1_BGDATA2 GH_CHIP_PPG_DEBUG2_DRE GH_CHIP_CAP_RAWDATA GH_CHIP_PPG_DC_INFO	Select what type of chip data the HR algorithm requires. By default, only the chip's PPG data is required.
GH_SPO2_DATA_TYPE	Defaults: GH_CHIP_PPG_DATA0 Optional: GH_CHIP_PPG_DATA1 GH_CHIP_PPG_DEBUG0 GH_CHIP_PPG_DEBUG1_BGDATA0 GH_CHIP_PPG_DEBUG1_BGDATA1 GH_CHIP_PPG_DEBUG1_BGDATA2 GH_CHIP_PPG_DEBUG2_DRE GH_CHIP_CAP_RAWDATA GH_CHIP_PPG_DC_INFO	Select what type of chip data the SPO2 algorithm requires. By default, only the chip's PPG data is required.
GH_HRV_DATA_TYPE	Defaults: GH_CHIP_PPG_DATA0	Select what type of chip data the HRV algorithm requires. By default, only the chip's PPG data is required.

Goodix Confidential
Goodix Confidential For lierda_ldl@lierda.com Only

	Optional: GH_CHIP_PPG_DATA1 GH_CHIP_PPG_DEBUG0 GH_CHIP_PPG_DEBUG1_BGDATA0 GH_CHIP_PPG_DEBUG1_BGDATA1 GH_CHIP_PPG_DEBUG1_BGDATA2 GH_CHIP_PPG_DEBUG2_DRE GH_CHIP_CAP_RAWDATA GH_CHIP_PPG_DC_INFO	
GH_NADT_DATA_TYPE	Defaults: GH_CHIP_PPG_DATA0 Optional: GH_CHIP_PPG_DATA1 GH_CHIP_PPG_DEBUG0 GH_CHIP_PPG_DEBUG1_BGDATA0 GH_CHIP_PPG_DEBUG1_BGDATA1 GH_CHIP_PPG_DEBUG1_BGDATA2 GH_CHIP_PPG_DEBUG2_DRE GH_CHIP_CAP_RAWDATA GH_CHIP_PPG_DC_INFO	Select what type of chip data the NADT algorithm requires. By default, only the chip's PPG data is required.
GH_TEST_DATA_TYPE	Defaults: GH_CHIP_PPG_DATA0 Optional: GH_CHIP_PPG_DATA1 GH_CHIP_PPG_DEBUG0 GH_CHIP_PPG_DEBUG1_BGDATA0 GH_CHIP_PPG_DEBUG1_BGDATA1 GH_CHIP_PPG_DEBUG1_BGDATA2 GH_CHIP_PPG_DEBUG2_DRE GH_CHIP_CAP_RAWDATA GH_CHIP_PPG_DC_INFO	Select what type of chip data the TEST algorithm requires. By default, only the chip's PPG data is required.
Note: If the algorithm requires a variety of different data, simply add the macros together		
End		

Goodix Confidential
Goodix Confidential For lierda_hl@lierda.com Only

fusion module configuration		
GH_FUSION_MODE_SEL	Defaults: GH_FUSION_MODE_ASYNC Optional: GH_FUSION_MODE_NONE GH_FUSION_MODE_SYNC GH_FUSION_MODE_ASYNC	The data synchronization mode of the fusion module can be selected in synchronous mode or asynchronous mode. If GH_FUSION_MODE_NONE is selected, this function will be removed.
GH_FIFO_READ_MAX_NUM	Defaults:255 Optional:1~255	The maximum number of fifo data packets that the SDK supports reading in a single interrupt
GH_ASYNC_GSENSOR_QUEUE_DEPTH	Defaults:1.5*100 Optional:According to actual application, it is recommended that the sampling rate should not exceed 1.5s*gsensor	The maximum Gsensor data depth supported by the Gsensor data asynchronous fusion solution
GH_SYNC_GSENSOR_DATA_MAX	Defaults: 1.5*100 Optional: According to actual application, it is recommended that the sampling rate should not exceed 1.5s*gsensor	The maximum Gsensor data depth supported by the Gsensor data synchronization fusion solution
GH_GSENSOR_DEBUG_EN	Defaults:0 Optional:1	1 means enabling gsensor debug of fusion module; 0 means disabling gsensor debug of fusion module
End		
MTSS(Mutil-sensor) Configuration		
GH_MTSS_EN	Defaults:1 Optional:0	1 means that the wearing logic of the SDK is required, enabling the multi-sensor data combination function to output action events; 0 means that the wearing logic of the SDK is not required;
GH_MOVE_GH_WEAR_ON_ITVAL_TIME	Defaults:3000 Optional:Set according to actual needs	Set the effective interval between motion events and wearing events, that is, an ADT wearing event is considered valid only if it occurs within the specified time after motion is detected.
End		
G-sensor&move detect configuration		
GH_GYRO_EN	Defaults:1 Optional:0	1 means gyroscope data is needed; 0 means gyroscope data is not needed;
GH_GS_MOVE_DET_EN	Defaults:1 Optional:0	1 means enabling the G-sensor motion detection function; 0 means disabling the G-sensor motion detection function;
GH_MOVE_DET_GS_SAMPLE_PERIOD_MS	Defaults:40	G-sensor sampling period, unit: ms
GH_GS_TS_INTERVAL_TOO_BIG_MS	Defaults:1000	The maximum interval between the timestamps before and after the G-sensor data. If the interval is exceeded, it is considered as a timestamp anomaly.
GH_GS_DIFF_THRD	Defaults:65*65	G-sensor motion detection threshold

Goodix Confidential For hierda_hd@hierda.com Only

GH_GS_MOVE_CNT_THRD	Defaults:2	G-sensor motion detection condition threshold. Motion is considered valid only if it exceeds this threshold continuously.
GH_GS_NOT_MOVE_CNT_THRD	Defaults:40	G-sensor still motion detection condition threshold, only when it exceeds this threshold continuously is it considered stillness valid
GH_GS_MOVE_TIME_OUT_THRD	Defaults:5000	G-sensor motion detection timeout, unit: ms
End		
Goodix protocol packaging configuration		
GH_PROTOCOL_EN	Defaults:0 Optional:1	1 means enabling Goodix standard protocol packaging; 0 means disabling Goodix standard protocol packaging; Goodix standard protocol is used to communicate with Goodix PC tool
End		
MIPS Statistics Module Configuration		
GH_APP_MIPS_STA_EN	Defaults:0 Optional:1	1 means that the MIPS statistics module is required; 0 means that the MIPS statistics module is not required; The MIPS statistics provided by the SDK are mainly used for the statistics of algorithm mips
GH_APP_MAIN_FREQUENCY_MHZ	Defaults:64	The main frequency of the current SDK running platform, unit: Mhz
GH_APP_TIMESTAMP_UNIT_US	Defaults:1000	The unit of the timestamp input to the MIPS module is defined in multiples of microseconds
End		
GH_LOG_DEBUG_ENABLE	Defaults:1 Optional:0	1 means turning on the SDK log function; 0 means turning off the SDK log function; Turning on this function helps users debug in the early stages, turning off this function can save ROM space

3.2.3 Add User Code

The code that user needs to add is stored in the "gh_app_user.c" file, and the API of this part is divided into the following categories.

3.2.3.1 Dynamic Memory Malloc and Free Function

If user enables GH_USE_DYNAMIC_ALGO_MEM, the dynamic memory malloc and free marco need to be implemented.

Table 3-10 Memory Handling Marco

Marco Name	Description	Parameter in		Parameter out	Return
GH_MALLOC	Memory malloc function	uint32_t unMemSize	Memory pool size	None	Memory address
GH_FREE	Memory free function	void* MemAddr	Memory pool address	None	None

Goodix Confidential For lierda_test@lierda.com Only

3.2.3.2 Timestamp function

If the user needs to use the mips module of the app layer, the function content of the interface for obtaining the app timestamp needs to be implemented.

Table 3-11 APP Timestamp function

Marco Name	Description	Parameter in	Parameter out	Return
gh_demo_app_mips_ts_get	APP layer obtains timestamp	None	None	timestamp

3.2.4 Call G-sensor(ACC+GYRO) Related API

3.2.4.1 Related API

This routine does not include the initialization of the G-sensor chip and the interaction process between the MCU and the G-sensor. The G-sensor data acquisition scheme is designed and implemented by the user. ACC recommended configuration in G-sensor: range ≥ ±4G, unit (512LSB/G), sampling rate (>= 25Hz).

3.2.4.2 Synchonous Read Mode

Set GH_FUSION_MODE_SEL to GH_FUSION_MODE_SYNC use Synchonous Read Mode. In this mode, it is required to synchronously read the G-sensor data and GH SDK data. Users can read the G-sensor data in function “gh_hal_fifo_data_publish” and push the G-sensor data to the fusion module by calling function “gh_demo_gsensor_data_sync_set”.

Table 3-12 Synchonous Read Mode API

Function Name	Description	Parameter in	Parameter out	Return
gh_demo_gsensor_data_sync_set	Push the G-sensor data to the fusion module	gh_gsensor_ts_and_data_t *p_data : point to G-sensor data buffer uint16_t size : Data size	None	Error code

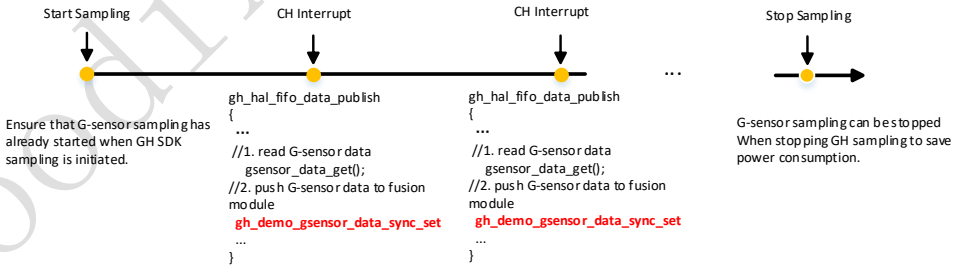


Figure 3-7 G-sensor API Synchronous Call Timing

If the user requires motion detection functionality, a timer needs to be implemented, and call function “gh_demo_gsensor_data_sync_set” when the timer ends.

Table 3-13 Motion Detection Timer API

Function Name	Description	Parameter in	Parameter out	Return
gh_move_det_timer_start	Start timer	None	None	Error code
gh_move_det_timer_stop	Stop timer	None	None	Error code

Goodix Confidential For lierda_lierda.com Only

3.2.4.3 Asynchronous Read Mode

Set GH_FUSION_MODE_SEL to GH_FUSION_MODE_ASYNC use Asynchonous Read Mode. In this mode, G-sensor data reading does not need to follow GH data. Each time G-sensor data is read, simply call the function “gh_demo_gsensor_data_set” to push the G-sensor data to the fusion module.

Function Name	Description	Parameter in	Parameter out	Return
gh_demo_gsensor_data _set	Push the G-sensor data to the fusion module	gh_gsensor_ts_and_data_t *p_data : point to G-sensor data buffer	None	Error code

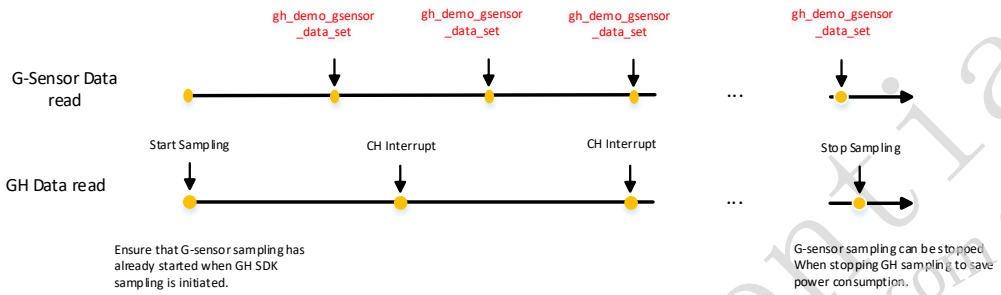


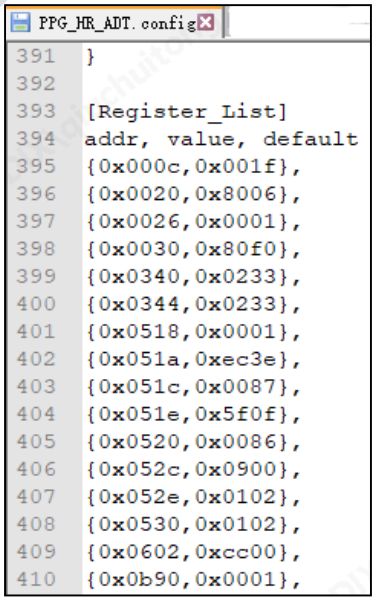
Figure 3-8 G-sensor API Asynchronous Call Timing

3.2.5 APP Layer Integration

If GH_USE_SDK_APP is set to 1, users can control the GH chip's operation by calling the APP-layer code without needing to directly call the HAL-layer code.

3.2.5.1 Add Configuration Table

The configuration table is at the end of the configuration file generated by of GHealth EVK tool. Copy the configuration table into an array.



```
PPG_HR_ADT.config
}

[Register_List]
addr, value, default
{0x000c,0x001f},
{0x0020,0x8006},
{0x0026,0x0001},
{0x0030,0x80f0},
{0x0340,0x0233},
{0x0344,0x0233},
{0x0518,0x0001},
{0x051a,0xec3e},
{0x051c,0x0087},
{0x051e,0x5f0f},
{0x0520,0x0086},
{0x052c,0x0900},
{0x052e,0x0102},
{0x0530,0x0102},
{0x0602,0xcc00},
{0x0b90,0x0001},
```

Figure 3-9 Register List

In order to simplify the operation of the GH SDK registers, user can use the EVK tool to generate the SDK register

Goodix Confidential For hierda_hd@hierda.com Only

configuration table, which stores all registers that need to be configured for the GH SDK and the required value written. The configuration table is provided to the user in the form of a register array, and is automatically loaded into the GH SDK in the routine.

User can copy the content of the register array generated by the tool to the "gh_reg_config.c" file.

```
/**
 * @brief default registers map
 */
const gh_config_reg_t g_reg_map[] =
{
    /* adt_hr_spo2_nadt_hrv */
    {0x0016,0x001f},// FASTEST_SAMPLE_RATE_DIVIDER:31,
    {0x0020,0x2978},// FIFO_WATER_LINE:120, RG_FIFO_READ_INT_TIMER:0.4s,
    {0x0380,0x3210},// RG_SLOT_INDEX0:0 : ppg_cfg_0, RG_SLOT_INDEX1:1 : ppg_cfg_1, RG_SLOT_INDEX2:2 : ppg_cfg_2, RG_SLOT_INDEX3:3 : ppg_cfg_3,
    {0x0382,0xf654},// RG_SLOT_INDEX4:4 : ppg_cfg_4, RG_SLOT_INDEX5:5 : ppg_cfg_5, RG_SLOT_INDEX6:6 : ppg_cfg_6, RG_SLOT_INDEX7:15: close,
    {0x038a,0x0093},// cfg0 SAMPLE_RATE_DIVIDER:9, cfg0 FIFO_CTRL:0, cfg0 RX_EN:1,
    {0x038c,0x0273},// cfg1 SAMPLE_RATE_DIVIDER:39, cfg1 FIFO_CTRL:0, cfg1 RX_EN:1,
    {0x038e,0x0273},// cfg2 SAMPLE_RATE_DIVIDER:39, cfg2 FIFO_CTRL:0, cfg2 RX_EN:1,
    {0x0390,0x0273},// Patch reg lsb:0 msb:1 val:3,
    {0x0392,0x0271},// cfg4 SAMPLE_RATE_DIVIDER:39, cfg4 FIFO_CTRL:0, cfg4 RX_EN:1,
    {0x0394,0x0271},// cfg5 SAMPLE_RATE_DIVIDER:39, cfg5 FIFO_CTRL:0, cfg5 RX_EN:1,
    {0x0396,0x0c71},// cfg6 SAMPLE_RATE_DIVIDER:199, cfg6 FIFO_CTRL:0, cfg6 RX_EN:1,
    {0x0398,0x0003},// Patch reg lsb:0 msb:1 val:3,
    {0x03a2,0x0001},// cfg4 RX1_PD_SEL:,
    {0x03a4,0x0001},// cfg5 RX1_PD_SEL:,
    {0x03a6,0x0001},// cfg6 RX1_PD_SEL:,
    {0x03aa,0x4443},// cfg0 RX0_TIA:250k, cfg0 RX1_TIA:250k, cfg0 BG CANCEL ENABLE:0, cfg0 DC CANCEL ENABLE:0, cfg0 ADC_INT:39.4us,
    {0x03ac,0x4443},// cfg1 RX0_TIA:250k, cfg1 RX1_TIA:250k, cfg1 BG CANCEL ENABLE:0, cfg1 DC CANCEL ENABLE:0, cfg1 ADC_INT:39.4us,
    {0x03ae,0x1147},// cfg2 RX0_TIA:25k, cfg2 RX1_TIA:25k, cfg2 BG CANCEL ENABLE:0, cfg2 DC CANCEL ENABLE:0, cfg2 ADC_INT:157.5us,
    {0x03b0,0x1147},// cfg3 RX0_TIA:25k, cfg3 RX1_TIA:25k, cfg3 BG CANCEL ENABLE:0, cfg3 DC CANCEL ENABLE:0, cfg3 ADC_INT:157.5us,
    {0x03b2,0x3242},// cfg4 RX0_TIA:50k, cfg4 RX1_TIA:100k, cfg4 BG CANCEL ENABLE:0, cfg4 DC CANCEL ENABLE:0, cfg4 ADC_INT:29.5us,
    {0x03b4,0x3203},// cfg5 BG_LEVEL:0 level, cfg5 RX0_TIA:50k, cfg5 RX1_TIA:100k, cfg5 BG CANCEL ENABLE:0, cfg5 DC CANCEL ENABLE:0, cfg5 ADC_IN
    {0x03b6,0x3242},// cfg6 RX0_TIA:50k, cfg6 RX1_TIA:100k, cfg6 BG CANCEL ENABLE:0, cfg6 DC CANCEL ENABLE:0, cfg6 ADC_INT:29.5us,
    {0x03da,0x1120},// cfg0 SLOT_POST_DELAY:rg_t3_set1, cfg0 *LED_SELECT:LED_0,
    {0x03dc,0x2420},// cfg1 SLOT_POST_DELAY:rg_t3_set1, cfg1 *LED_SELECT:LED_1,
    {0x03de,0x1200},// cfg2 *LED_SELECT:LED_2,
    {0x03e0,0x1000},// cfg3 *LED_SELECT:LED_4,
    {0x03e2,0x1020},// cfg4 SLOT_POST_DELAY:rg_t3_set1, cfg4 *LED_SELECT:LED_4,
    {0x03e4,0x0020},// cfg5 SLOT_POST_DELAY:rg_t3_set1, cfg5 *LED_SELECT:,
    {0x03e6,0x1020},// cfg6 SLOT_POST_DELAY:rg_t3_set1, cfg6 *LED_SELECT:LED_4,
    {0x03ea,0x0020},// cfg0 DRV0_CURRENT(mA):20.0784, cfg0 DRV1_CURRENT(mA):0,
    {0x03ec,0x0020},// cfg1 DRV0_CURRENT(mA):20.0784, cfg1 DRV1_CURRENT(mA):0,
    {0x03ee,0x0040},// cfg2 DRV0_CURRENT(mA):40.1569, cfg2 DRV1_CURRENT(mA):0,
    {0x03f0,0x0040},// cfg3 DRV0_CURRENT(mA):40.1569, cfg3 DRV1_CURRENT(mA):0,
    {0x03f2,0x0010},// cfg4 DRV0_CURRENT(mA):10.0392, cfg4 DRV1_CURRENT(mA):0,
    {0x03f6,0x0010},// cfg6 DRV0_CURRENT(mA):10.0392, cfg6 DRV1_CURRENT(mA):0,
    {0x0484,0x0001},// RG_ADC_XP_SL:0:3.25MHz,
    {0x0488,0x022f},// LED_DRV0_FS:160mA, LED_DRV1_FS:160mA,
    {0x0496,0x0ccd},// RG_DRE_DC_THR:3277, RG_DRE_DC_THR_CNT:1,
    {0x049a,0x00b0},// RG_T3_SET1:80us,
    {0x0500,0x000b},// FIFO_UP_OVERFLOW_EN:1, FIFO_DOWN_OVERLOWE EN:0, FIFO_WATERLINE_EN:1, TIMER_INTERRUPT_EN:0, USER_INTERRUPT_EN:0, FRAME_DON
    {0x1000,0x0002},// work mode:2,
```

Figure 3-10 Add Configuration Files to Routines

The above figure is a configuration table that comes with the routine, and user can replace it with a new configuration table.

3.2.5.2 GH SDK Main Process

1. When the main program starts, initialize SDK, download configuration, and start sampling.

2.If GH_ISR_MODE is set to INTERRUPT_MODE, wait for the sensor chip to trigger an interrupt, and then handle the interrupt in the main loop.

3. If GH_ISR_MODE is set to POLLING_MODE, Poll and call the relevant functions to handle GH chip events.

4. Call the relevant functions to turn off sampling as needed.

5. In Asynchronous Read Mode, the G-sensor task independently reads G-sensor data and pushes it to the GH SDK.

6. The functions in the APP layer are not thread-safe and require locking when called in multi-threaded scenarios.

Goodix Confidential For lierda_hd@lierda.com Only

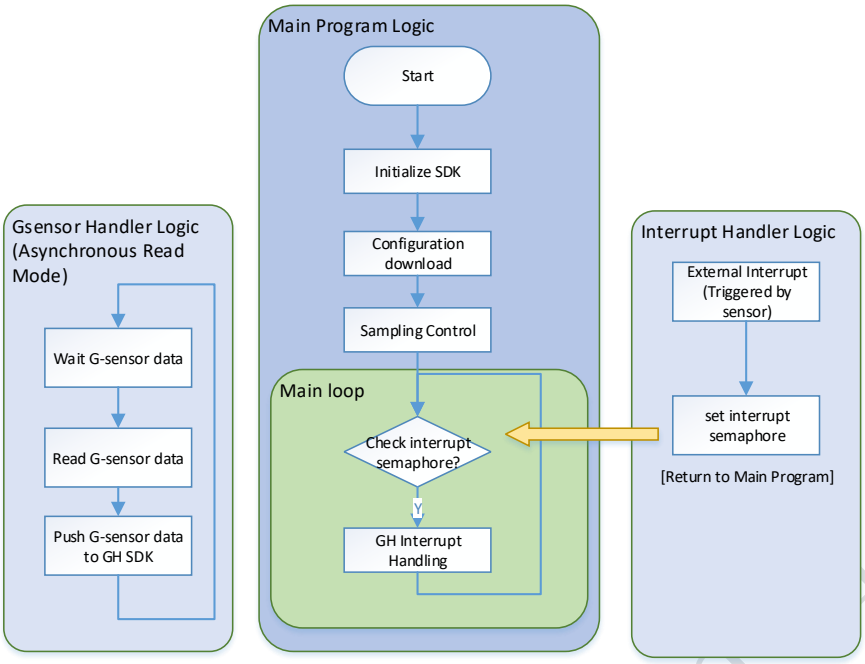


Figure 3-11 Main Process(Interrupt mode)

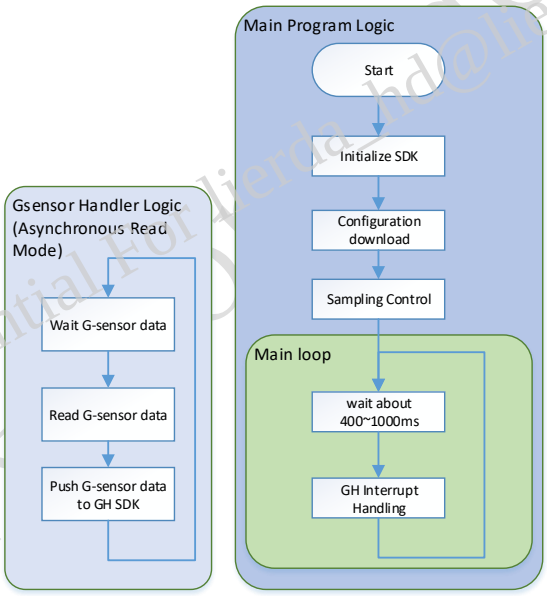


Figure 3-12 Main Process(Polling mode)

3.2.5.3 Initialize API Call

1. Call " gh_demo_init ()" when initializing GH SDK APP, If the return value is SUCCESS, the initialization of the SDK APP can be completed.
2. Call “gh_hal_service_init ()” when initializing GH SDK HAL, If the return value is SUCCESS, the initialization of the SDK HAL can be completed.

Note:

During the call to “gh_hal_service_init ()”, the API will internally complete the initialization of functions such as the SPI/I2C API, GH SDK, and IO pins. Since the routine code does not include the initialization of the G-sensor, the user needs to initialize the G-sensor module separately.

Goodix Confidential for lierda_hd@lierda.com Only

3.2.5.4 Configuration download

Call “gh_demo_config_write (gh_config_reg_t *p_cfg, uint16_t number)” to download registers.

Example: “gh_demo_config_write ((gh_config_reg_t *) g_reg_map, g_reg_map_szie)”, Users can get g_reg_map and g_reg_map_szie from gh_reg_config.c file.

3.2.5.5 Sampling Control

When starting GH chip sampling, call " gh_demo_function_ctrl ()". The formal parameter of the function represents the function that needs to be enabled. Currently, it supports the sampling of ADT, HR, SPO2, HRV, GNADT, IRNADT and SLOT functions. Multiple functions can be enabled at one time through the struct.

Example: enable ADT+GNADT+HR+SPO2+HRV at the same time.

```
gh_function_en_union_t func_en;

func_en.bits = 0;
func_en.func.function_adt_en = 1;
func_en.func.function_gnadt_en = 1;
func_en.func.function_hr_en = 1;
func_en.func.function_spo2_en = 1;
func_en.func.function_hrv_en = 1;
gh_demo_function_ctrl (&func_en);
```

When stopping GH chip sampling, call " gh_demo_function_ctrl ()". The function parameter represents the function that needs to be turned off, and multiple functions can be turned off at one time.

Example: disable GNADT+HR+SPO2+HRV at the same time, but keep ADT enable.

```
gh_function_en_union_t func_en;

func_en.bits = 0;
func_en.func.function_adt_en = 1;
func_en.func.function_gnadt_en = 0;
func_en.func.function_hr_en = 0;
func_en.func.function_spo2_en = 0;
func_en.func.function_hrv_en = 0;
gh_demo_function_ctrl (&func_en);
```

3.2.5.6 GH Interrupt Handling

If user uses polling mode to interact with GH chip, user needs to set "GH_ISR_MODE" to "POLLING_MODE", and at the same time, when the polling time comes, call “gh_demo_int_process ()” for processing.

If user needs to handle interrupt events, “gh_demo_int_process ()” needs to be called when a hardware interrupt is generated.

3.2.5.7 Data from Callback

After starting the sampling, the user can obtain data from the function “gh_demo_data_publish”. The specific data format can be found in the Appendix 1.

Goodix Confidential For lierda_hd@lierda.com Only

3.2.6 Threads Arrangement

Because "gh_demo_sample_ctrl", "gh_demo_int_process", "gh_demo_init", "gh_demo_config_write", “gh_demo_gsensor_data_set”, “gh_demo_gsensor_data_sync_set”, and “gh_hal_service_init” will use the internal functions of the GH SDK and involve the operation of the GH chip. Therefore, the above functions must be placed in one thread. If multiple threads are to be placed, they must be locked and mutually exclusive to prevent resource conflicts.

3.3 Porting Wear Detection

The GH SDK provides three methods for wear detection:

1. ADT: Uses infrared light to determine whether the device is worn. If the signal is above the "wear on" threshold, it is determined to be "wear on," and if the signal falls below the "wear off" threshold, it is determined to be "wear off."
2. Motion Detection: Determines whether there is movement based on gsensor data.
3. NADT Algorithm: Uses green light or infrared to determine if the current object being worn is a living entity.

The GH SDK provides the following wear detection flow example:

1. The initial state is "wear off." When ADT detects "wear on," it transitions to the "wear on" state.
2. In the "wear on" state, if ADT detects "wear off," it reverts to the "wear off" state.
3. In the "wear on" state, if the NADT algorithm determines that the object is non-living, it transitions to the "wear off (non-living)" state.
4. In the "wear off (non-living)" state, ADT must detect "wear on" and movement for it to transition back to the "wear on" state.

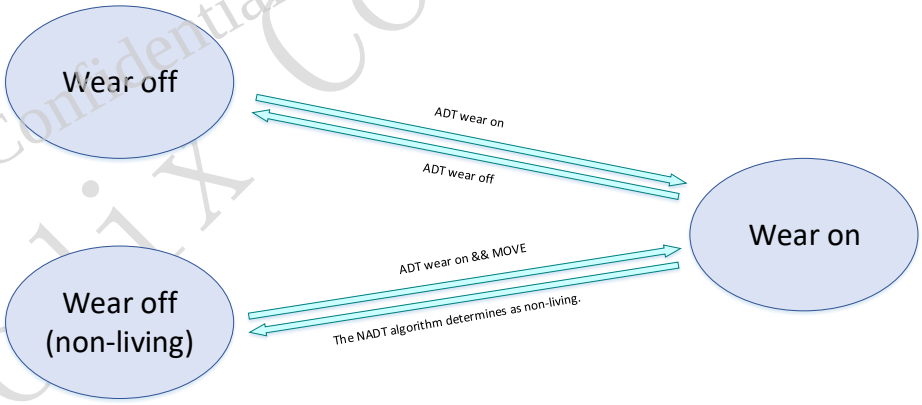


Figure 3-13 Wear Detection Status

3.3.1 Porting Steps

1. Set GH_MTSS_EN to 1 and set GH_GS_MOVE_DET_EN to 1.
2. Call the G-sensor related API as described in Section 3.2.5.
3. Complete function “gh_demo_mtss_event_publish”:

(1) Upon receiving the GH_ACTION_WEAR_ON event, start the main function (e.g., HR).
(2) Upon receiving the GH_ACTION_WEAR_OFF event, stop the all function and the start the ADT function.
(3) Upon receiving the GH_ACTION_MOVE event, start the ADT function.
(4) Upon receiving the GH_ACTION_MOVE_TIME_OUT event, stop the ADT function.

Goodix Confidential For lierda_hd@lierda.com Only

4. Start the ADT function after the program is initialized.

3.4 Porting Goodix Protocol

The GH SDK includes the Goodix protocol, which can be used to connect with the Goodix PC tool and the Goodix app. This protocol system facilitates debugging and data collection.

3.4.1 Porting Steps

1. Set GH_PROTOCOL_EN to 1.
2. Implement functions: “gh_protocol_lock”, “gh_protocol_unlock”, “gh_protocol_delay”, and “gh_protocol_data_send”.
3. Call “gh_protocol_init” during the initialization phase.
4. Call “gh_protocol_data_receive” when a command is received from the PC TOOL or APP.
5. Implement the functions "download_config" and "GH3X_SwFunctionCmd" in gh_protocol_cmd.c based on the demo code.

Note:

If the API “gh_protocol_data_receive” is called in different thread/task from other GH SDK API, please add lock/Mutex to prevent conflict about simultaneous running.

3.4.2 BLE peripheral setting

1. BLE profile relative to GOODIX APP need to be set as follows:

Servive UUID:”0000190e-0000-1000-8000-00805f9b34fb”

Data Tx characteristic, Master connects successfully and wait for recieiving data from BLE slave.

TX_UUID:”00000003-0000-1000-8000-00805f9b34fb”

Data Rx characteristic UUID, Master connects successfully and can transfer data to BLE slave.

RX_UUID:”00000004-0000-1000-8000-00805f9b34fb”

2. BLE mtu setting

Set BLE mtu size to 247.

Goodix Confidential
Goodix Confidential For lierda_hd@lierda.com Only

4 Advanced Feature Interfaces

To enable more flexible operation and control, the GH SDK offers a set of advanced interfaces.

Table 4-1 agc disable Function

Function Name	gh_service_agc_disable	
Description	Disable PPG AGC	
Parameter in	none	
Parameter out	none	
Return	Error code 0: success	

Table 4-2 LED current set Function

Function Name	gh_service_led_current_set	
Description	Set PPG LED current	
Parameter in	uint8_t slot_cfg_num	slot config num
	uint8_t drv_idx	LED driver index
	uint32_t current_ma	LED current(unit : mA)
Parameter out	none	
Return	Error code 0: success	

Table 4-3 PPG sample rate set Function

Function Name	gh_service_sample_rate_set	
Description	Set PPG slot sample rate	
Parameter in	uint8_t slot_cfg_num	slot config num
	uint16_t sample_rate	sample rate(unint : Hz)
Parameter out	none	
Return	Error code 0: success	

Table 4-4 TIA gain set Function

Function Name	gh_service_tia_gain_set	
Description	Set TIA gain.	
Parameter in	uint8_t slot_cfg_num	slot config num
	uint8_t rx_idx	RX channel index
	uint8_t gain_code	gain code
Parameter out	none	
Return	Error code 0: success	

Table 4-5 ADC integration time set Function

Function Name	gh_service_int_time_set	
Description	Set ADC integration time	
Parameter in	uint8_t slot_cfg_num	slot config num
	gh_hal_int_time_e int_time	ADC integration time

Goodix Confidential For lierda_hd@lierda.com Only

Parameter out	none
Return	Error code 0: success

Goodix Confidential For lierda_hd@lierda.com Only
Goodix Confidential

5 Appendix

5.1 Appendix 1: Frame Data Structure Definition

Table 5-1 Structure “gh_func_frame_t” member Description

Member Name	Type	Description
frame_cnt	uint32_t	frame count, start from 0
timestamp	uint32_t *	timestamp of this frame
gsensor_data	gh_gsensor_data_t	gsensor data
id	gh_func_fix_idx_e	function id
p_ch_map	gh_hal_data_channel_t*	Point to channel map which discribe the channel data type
p_data	gh_frame_data_t*	Point to channel data
ch_num	uint8_t	channel number
ch_max	uint8_t	channel max number
gsensor_en	uint8_t	gsensor enable flag
fifo_end_flag	uint8_t	fifo end flag, indecate the last frame of this fifo if 1
led_drv_fs	uint8_t array	LED driver full scale, unit: mA
p_algo_res	void *	Algorithm result, point to different algorithm result struct: HR: gh_algo_hr_result_t SPO2: gh_algo_spo2_result_t HRV: gh_algo_hrv_result_t GNADT: gh_algo_nadt_result_t IRNADT: gh_algo_nadt_result_t

Table 5-2 Structure “gh_frame_data _t” member Description

Member Name	Type	Description
ipd_pa	int32_t	unit: pA
rawdata	int32_t	raw data
flag	gh_frame_data_flag_t	flag
agc_info	gh_agc_info_t	agc info

Goodix Confidential For lerda_lt@lierda.com Only

5.2 Appendix 2: Troubleshooting FAQs During Porting

5.2.1 Compilation Error

Table 5-3 Common compilation errors

Phenomenon	Direction	Step
Compilation error after closing or opening some macros	Check whether it is a standard C function (such as malloc, free, snprintf); Check whether the current platform supports.	1. If malloc and free are not supported, please replace them with equivalent functions supported by the platform 2. If snprintf is not supported, please set GH_USE_STD_SNPRINTF to 0

5.2.2 Link Error

Table 5-4 Common link errors

Phenomenon	Direction	Step
The GH SDK takes up more RAM or Rom than expected	Some configuration macros may not be closed	Check each macro in config.h, close the functions that are not used in the current project, and set the buffer size reasonably
Link error after closing or opening some macros: Some functions in the driver or demo code cannot find entities	Whether the error is due to the link sequence	Try to link in the way of sampling static library groups, such as 8.2.a, syntax reference under the cmake tool
Link error after closing or opening some macros: __sbrk, __write, __close, __fstat, _isatty, __lseek, _read entity not found	The current platform may not support snprintf	The Log method use __EXAMPLE_LOG_METHOD_1__
	Unused functions may be linked in	Check whether there is a problem with the link option, such as whether the --gc-section option has been added under the cmake tool

Note:

Example 8.2.a cmaketool syntax reference

```
target_link_libraries(target_name
-Wl,--start-group
cortexM4_eabi-gcc-hard_common.a
cortexM4_eabi-gcc-hard_hr.a
-Wl,--end-group
)
```

Goodix Confidential For lierda_hu@lierda.com Only

5.2.3 Operation Exceptions

Table 5-5 operation Exceptions

Phenomenon	Direction	Step
Program is stuck in a function and cannot come out	Whether to stuck in a while(1)	Turn off the mcu watchdog Open the log to see if there is an error in the log Single step
GH SDK outputs the following log: "gh_hal_interface_init error "	No communication	Check whether the hardware connection and power supply are normal Use a logic analyzer to analyze the timing and compare it with the data on the Datasheet to confirm whether there is any abnormality in the timing
Probabilistic opening function failure or data abnormality	Whether the top-level API has mutual exclusion processing	The functions "gh_demo_sample_ctrl", "gh_demo_int_process", "gh_demo_init", "gh_demo_config_write" , "gh_demo_gsensor_data_set" , "gh_demo_gsensor_data_sync_set", "gh_hal_service_init", and "gh_hal_isr" should be executed within the same thread. If they cannot be executed in the same thread, proper locking mechanisms must be implemented to prevent parallel conflicts.
	Stack is not enough	Please confirm whether the Stack setting in the current working mode aligns with the stack requirements outlined in the resource statistics table of the current version. If the requirements are not met, kindly increase the stack accordingly.

Goodix Confidential For herta_ld@herta.com only
Goodix Confidential

5.3 Appendix 3: I2C Device Address Configuration

Table 5-6 I2C Address Configuration

Macro	MISO state	CS state	Device address (7bit)	Write address (8bit)	Read address (8bit)
GH_I2C_ID_SEL_1L0L	LOW	LOW	0x14	0x28	0x29
GH_I2C_ID_SEL_1L0H	LOW	HIGH	0x15	0x2A	0x2B
GH_I2C_ID_SEL_1H0L	HIGH	LOW	0x16	0x2C	0x2D
GH_I2C_ID_SEL_1H0H	HIGH	HIGH	0x17	0x2E	0x2F

Goodix Confidential For lierda_hd@lierda.com Only
Goodix Confidential